**TÀI LIỆU HƯỚNG DẪN XÂY DỰNG HỆ THỐNG ĐẾM NGƯỜI (YOLOv11 & BO-SORT)**

Mục Tiêu:

Người đi từ ngoài vào trong: tăng bộ đếm IN.

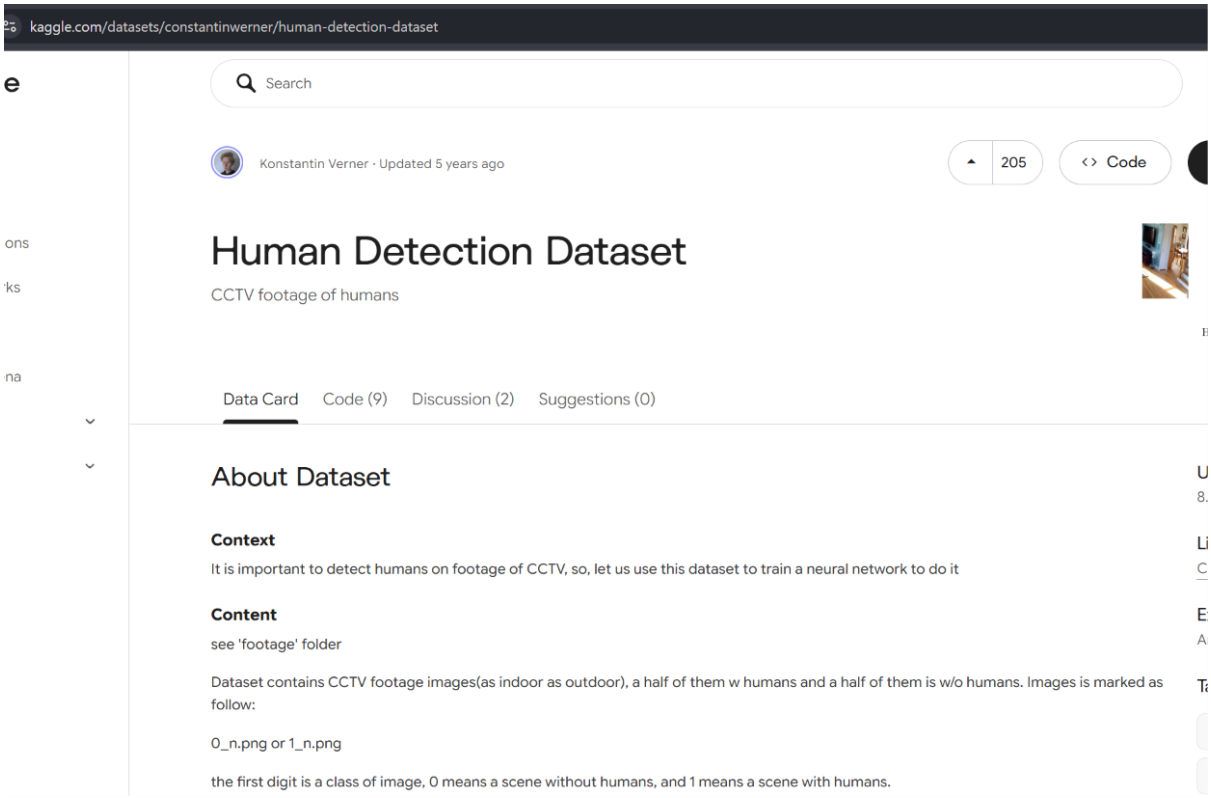
Người đi từ trong ra ngoài: tăng bộ đếm OUT.

Mỗi ID chỉ được ghi nhận một lần khi vượt qua đường đếm, tránh đếm lặp do người đứng gần cửa.

**GIAI ĐOẠN 1: THU THẬP DỮ LIỆU TỪ KAGGLE**

Kaggle là "mỏ vàng" dữ liệu miễn phí. Thay vì phải đi quay video hàng giờ đồng hồ, bạn sẽ lấy dữ liệu có sẵn.

1. **Tìm kiếm:** Truy cập kaggle.com. Tìm các từ khóa: *"Pedestrian detection YOLO"*, *"Person tracking dataset"*, hoặc *"Crowd counting"*.
2. **Lựa chọn:** Ưu tiên tải các bộ dữ liệu đã được gán nhãn sẵn định dạng **YOLO (file .txt)** hoặc **COCO (file .json)**.
3. **Tải & Giải nén:** Bấm Download tập tin .zip về máy và giải nén ra một thư mục (Ví dụ: D:\Kaggle_Data).



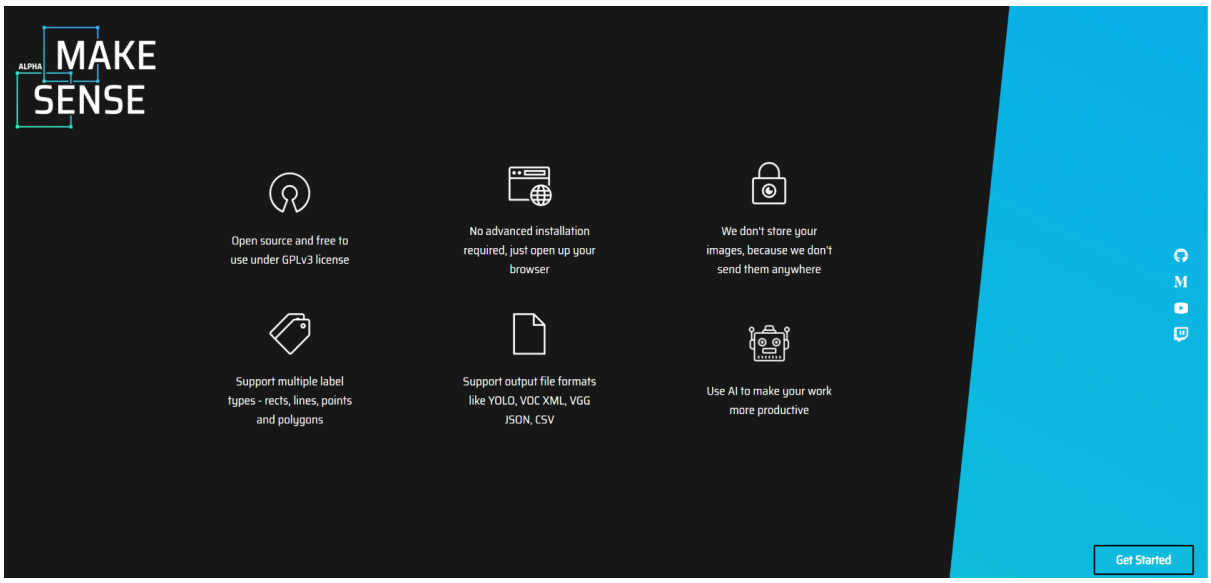

https://www.kaggle.com/datasets/constantinwerner/human-detection-dataset

## GIAI ĐOẠN 2: XỬ LÝ & GÁN NHÃN VỚI ROBOFLOW VÀ MAKESENSE

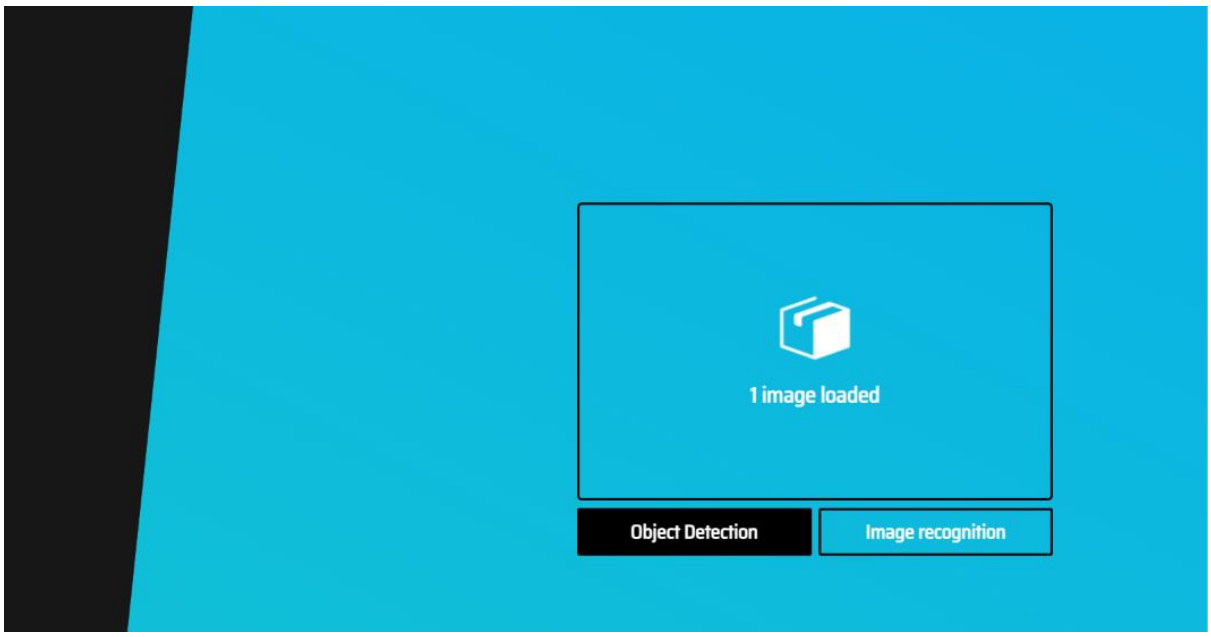
Dữ liệu tải trên mạng thường rất lộn xộn (chứa nhãn ô tô, chó mèo...). Chúng ta cần làm sạch để AI chỉ tập trung đếm "Người".

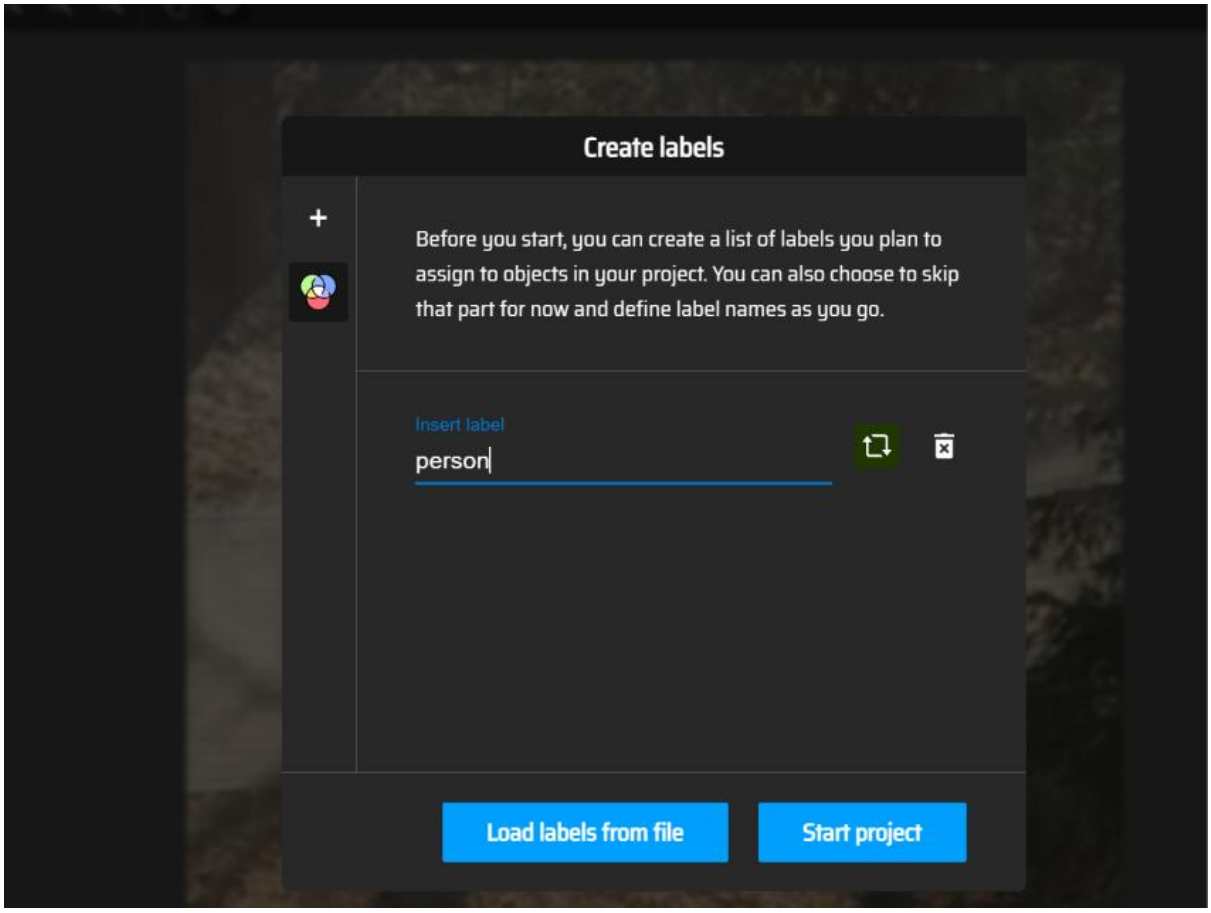
### Bước 2.1: Chữa cháy thủ công bằng MakeSense.ai (Nếu cần thiết)

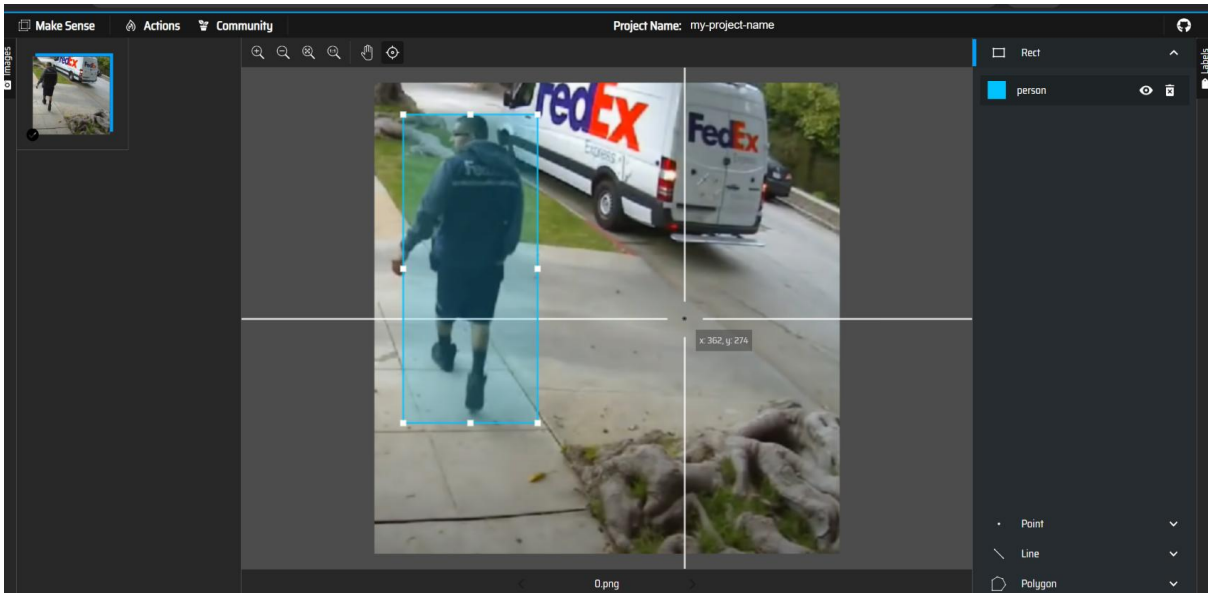
- Nếu bộ dữ liệu từ Kaggle có một vài ảnh chưa được gán nhãn, hãy mở trang web makesense.ai.: https://www.makesense.ai/

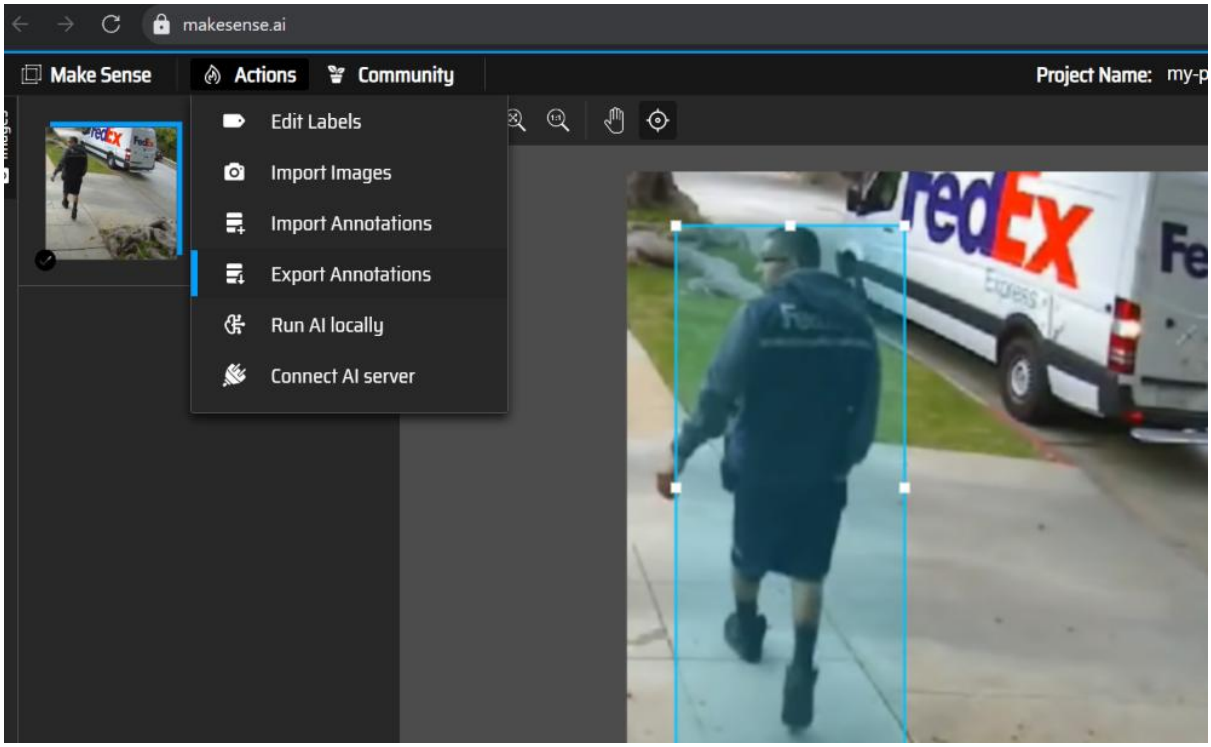


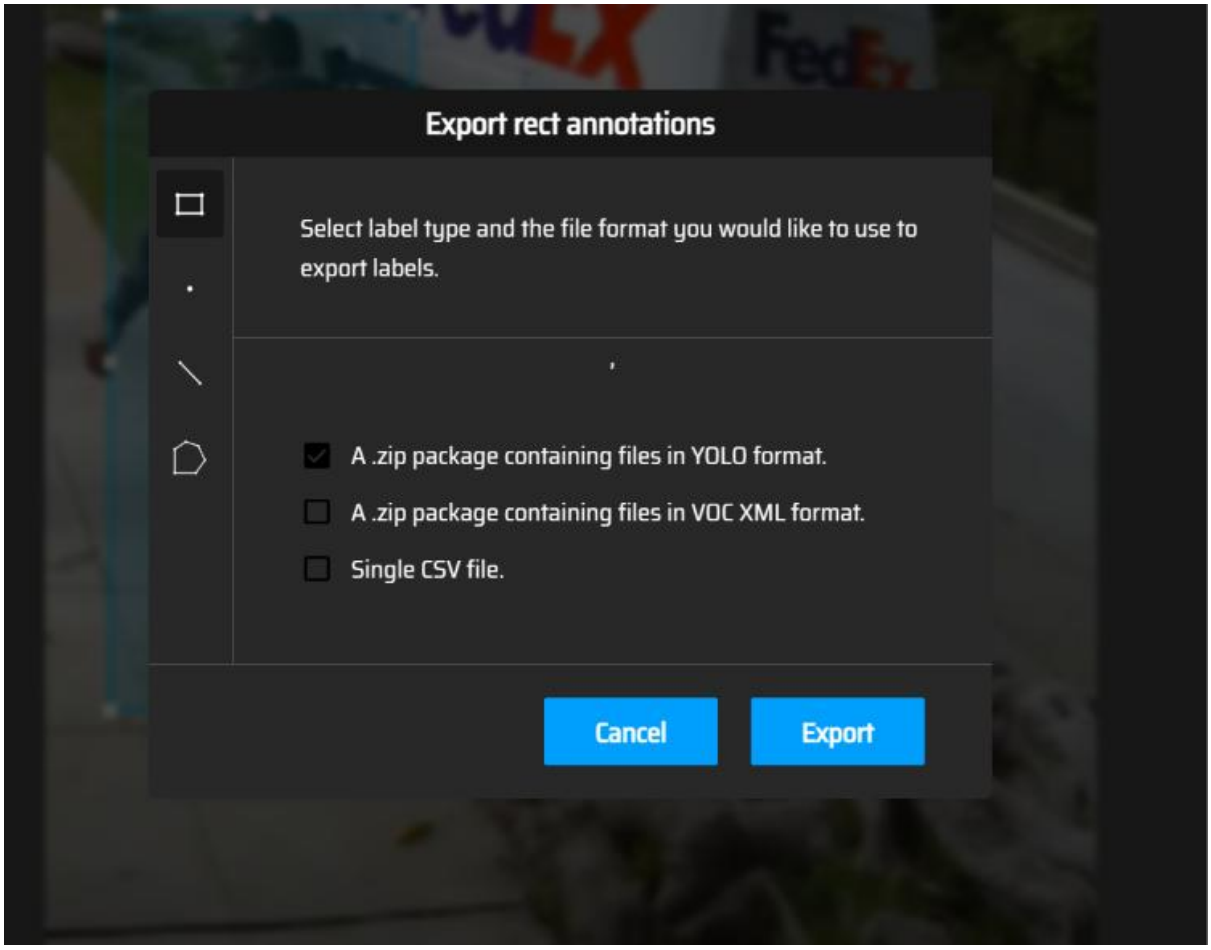

- Tải các ảnh đó lên, chọn chế độ *Object Detection*, tạo nhãn person và dùng chuột khoanh vùng thủ công. Sau đó xuất file .txt (YOLO format) tải về máy.




Create labels
Before you start, you can create a list of labels you plan to assign to objects in your project. You can also choose to skip that part for now and define label names as you go.
Insert label
person
Load labels from file
Start project

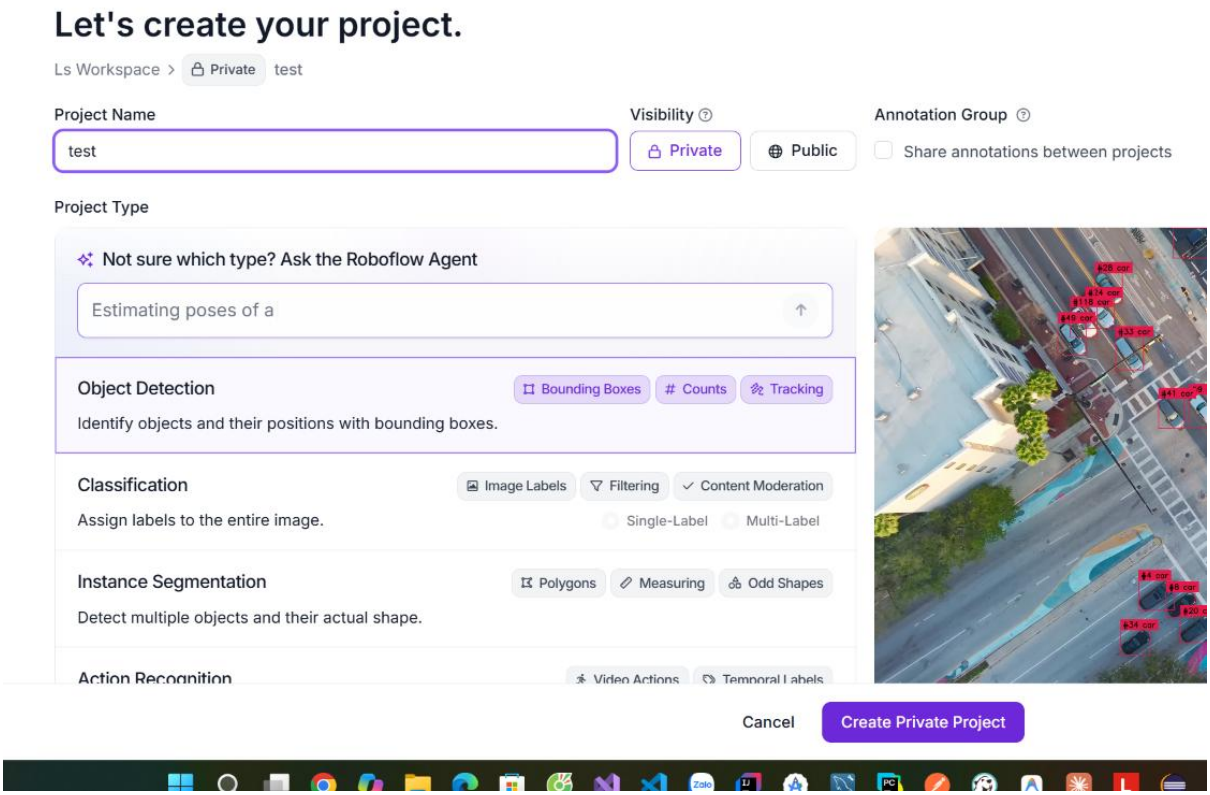

Make Sense
Actions
Community
Project Name: my-project-name
Rect
person
Point
Line
Polygon
0.png



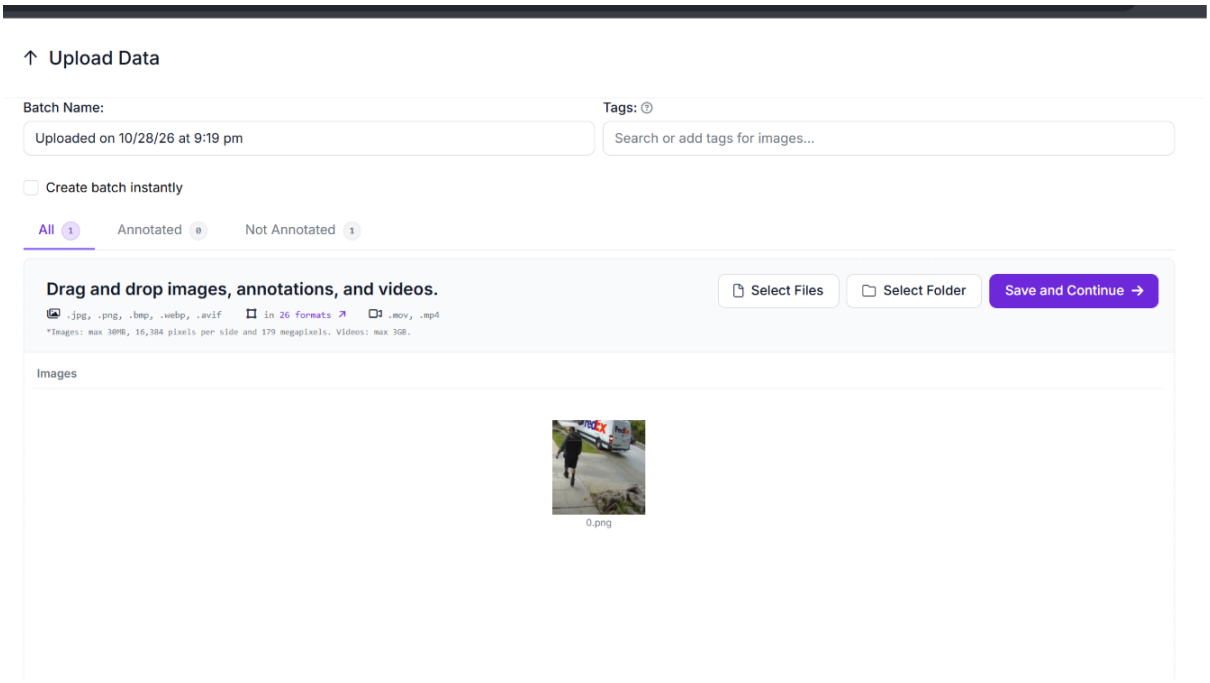




**Bước 2.2: Làm sạch và Tăng cường bằng Roboflow (tự động nhờ AI gán nhãn tự động)**

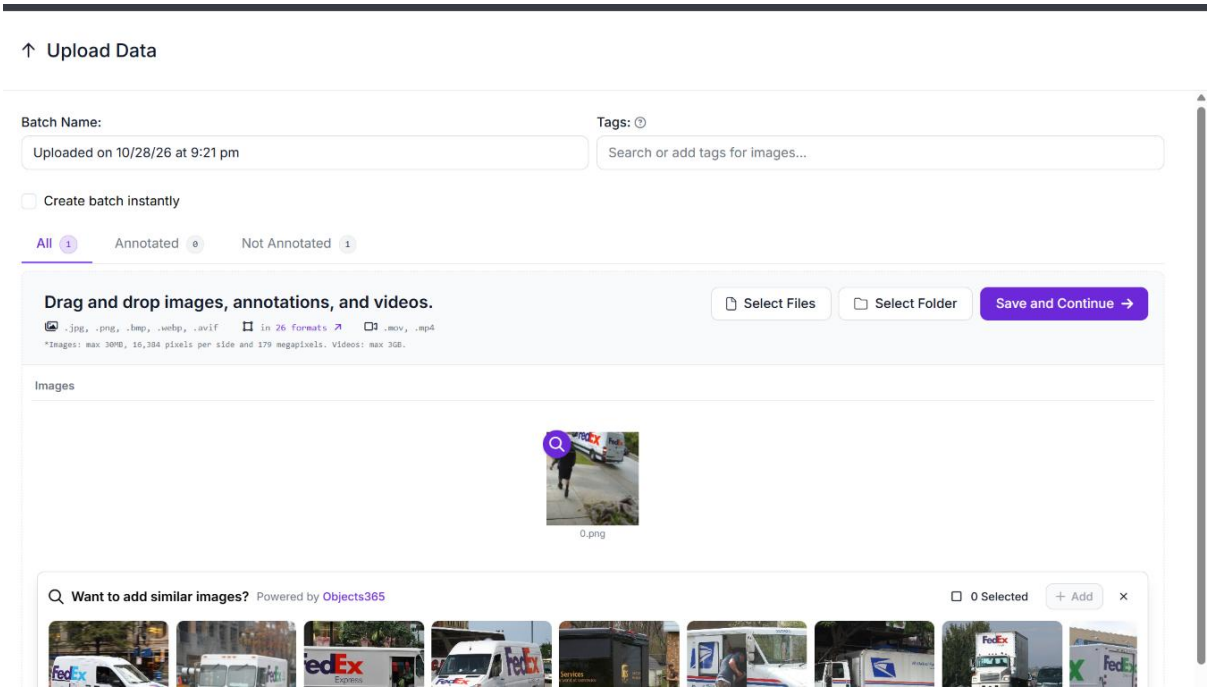
1. **Khởi tạo:** Tạo tài khoản Roboflow, tạo Project mới (Object Detection), mục tiêu nhận diện là person.



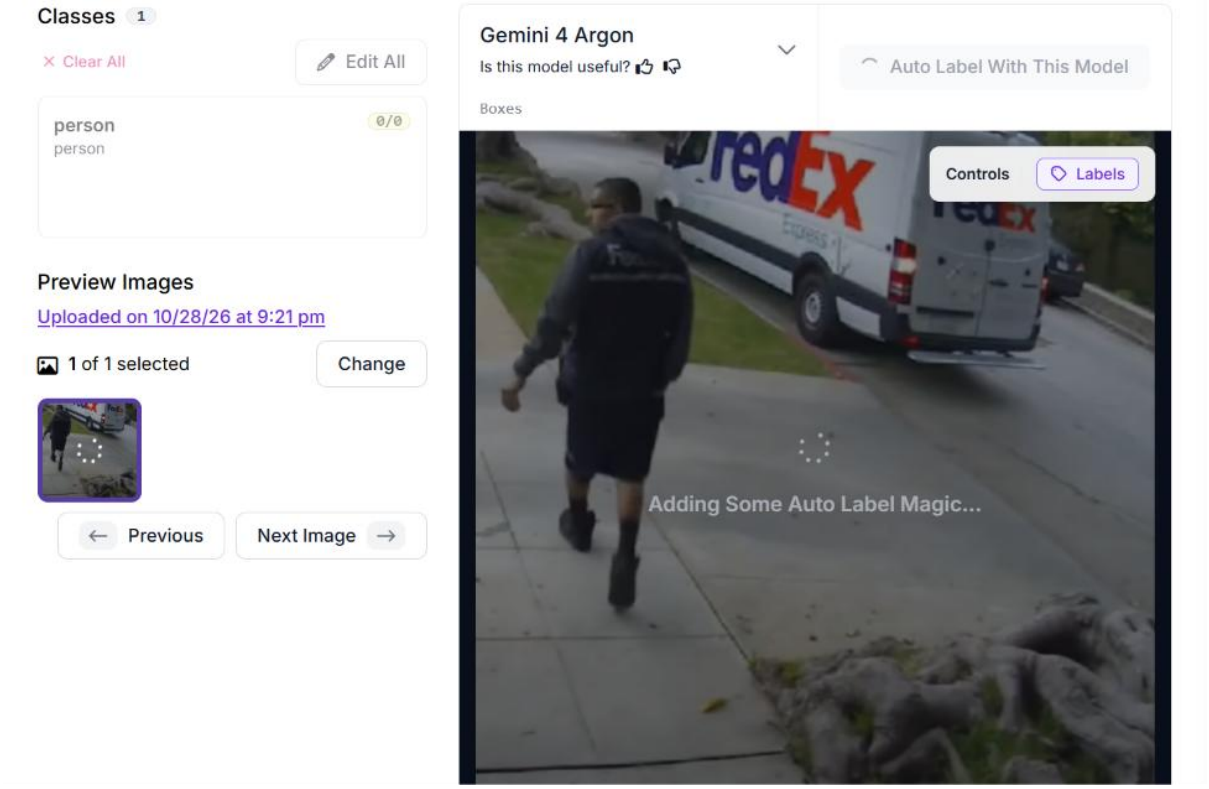

2. **Tải lên:** Kéo thả toàn bộ thư mục dữ liệu (gồm ảnh và file .txt) vào trình duyệt.

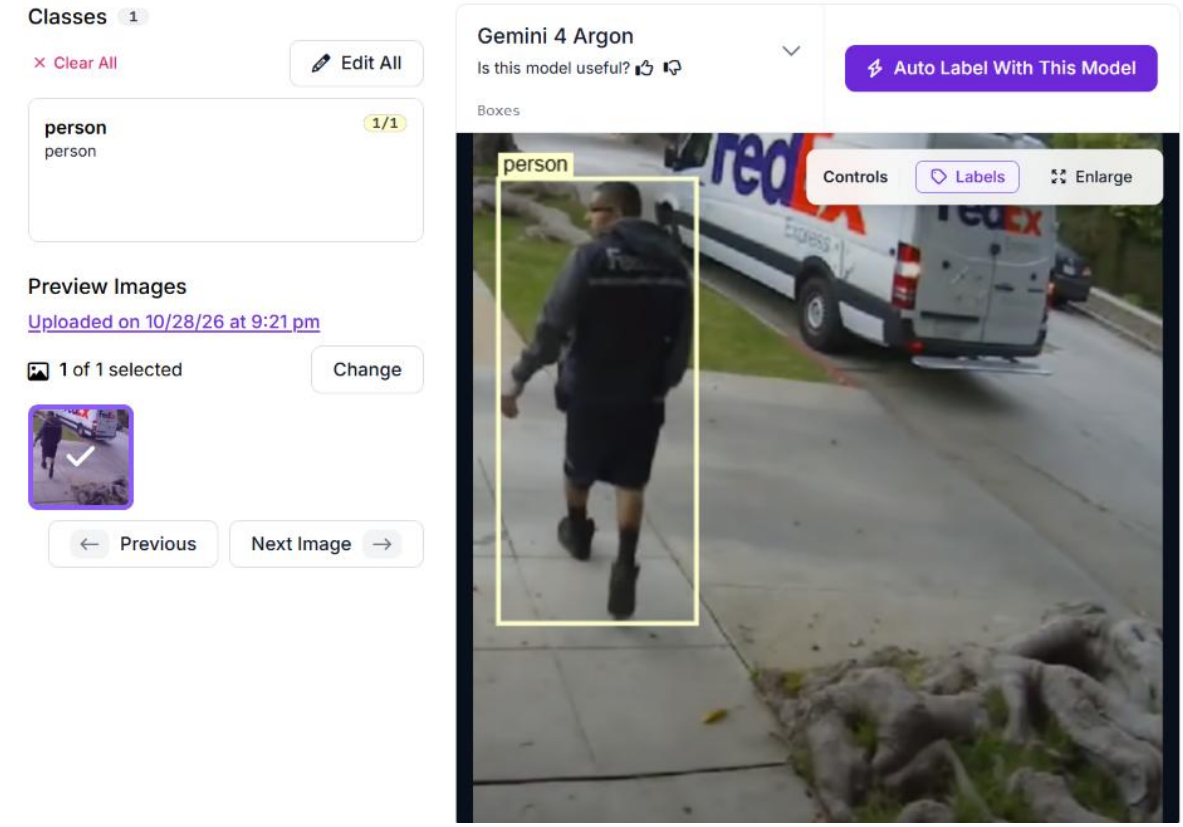



Bấm save and continue:

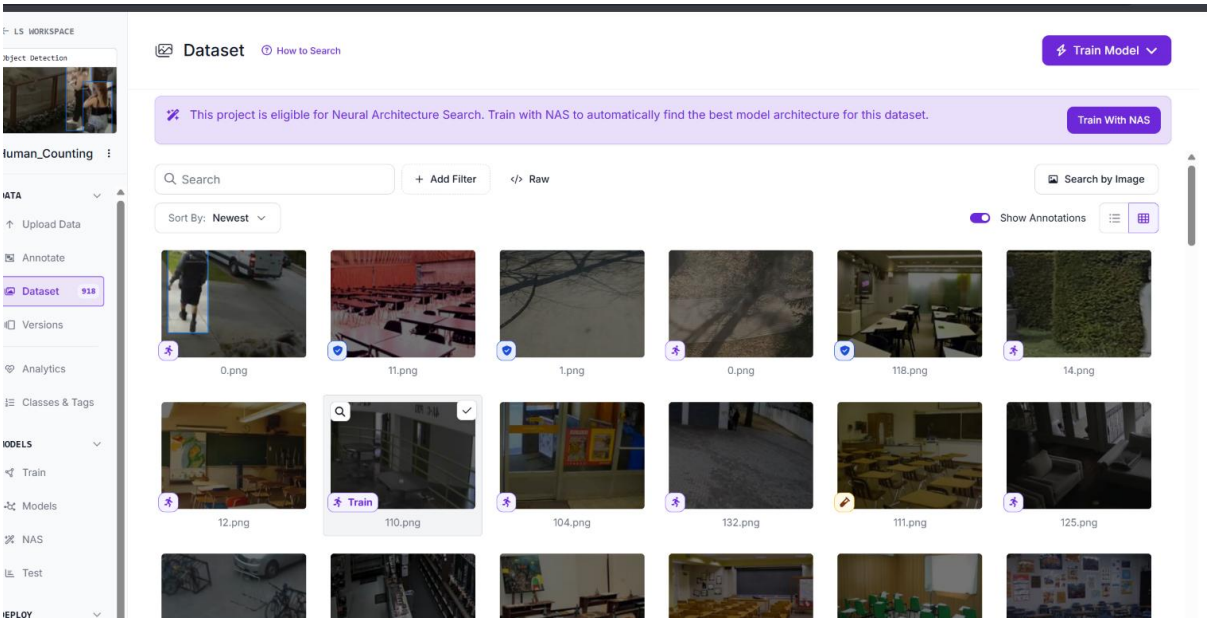



3. **Lọc nhãn (Filter):** Vào mục *Classes*, xóa bỏ toàn bộ các nhãn không phải con người (car, dog, bicycle...), chỉ giữ lại duy nhất class person. Và chọn AI gán nhãn hộ

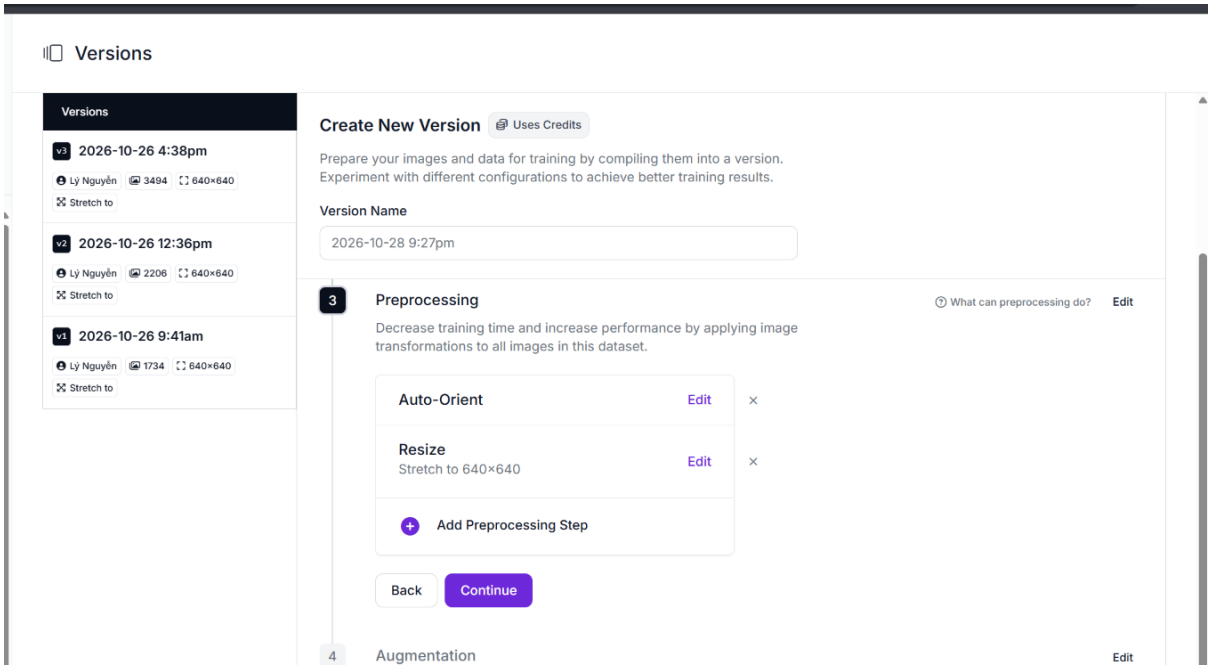




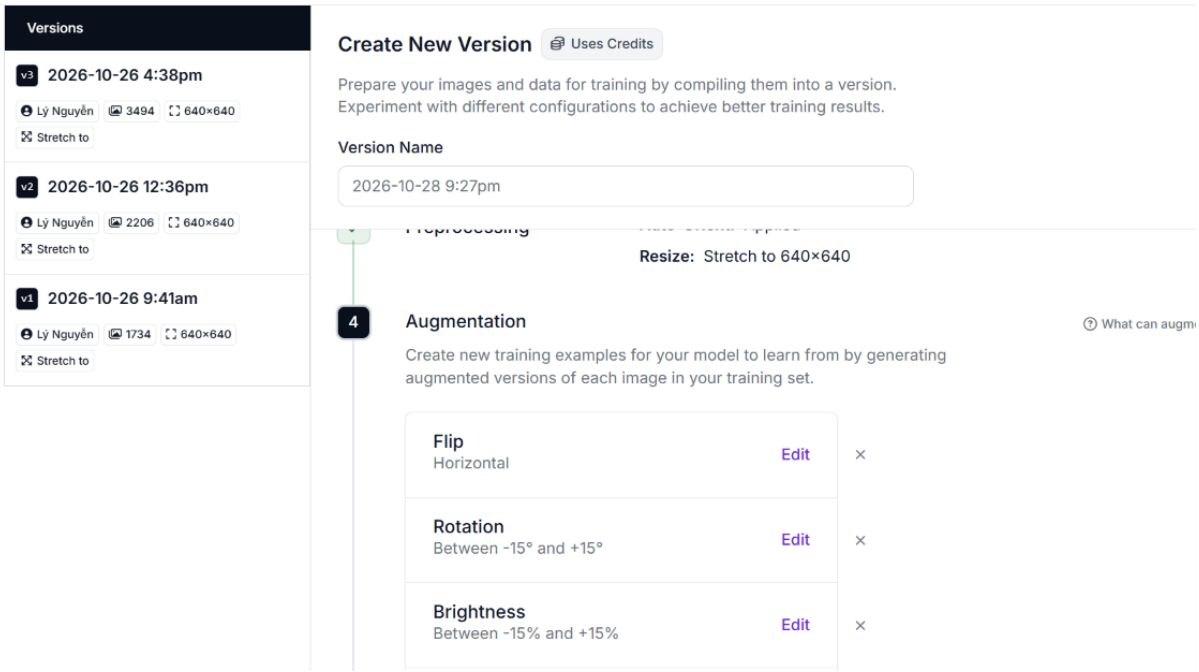

Sau khi tự động tạo bạn vào để duyệt do AI gán từng bức ảnh hoặc duyệt cùng 1 lúc và add vào data set.

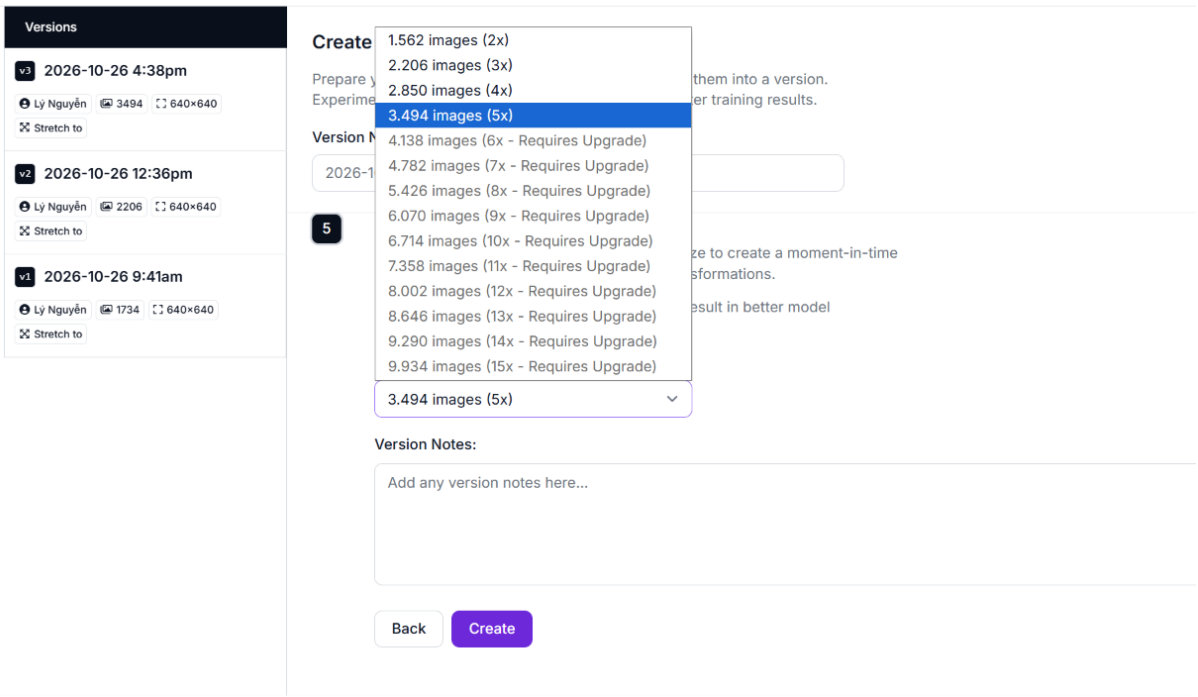



4. **Tiền xử lý (Preprocessing):** Thêm bước *Resize (Stretch to 640x640)*. Điều này ép mọi ảnh về chuẩn vuông 640x640, giúp quá trình train sau này nhẹ và nhanh hơn.

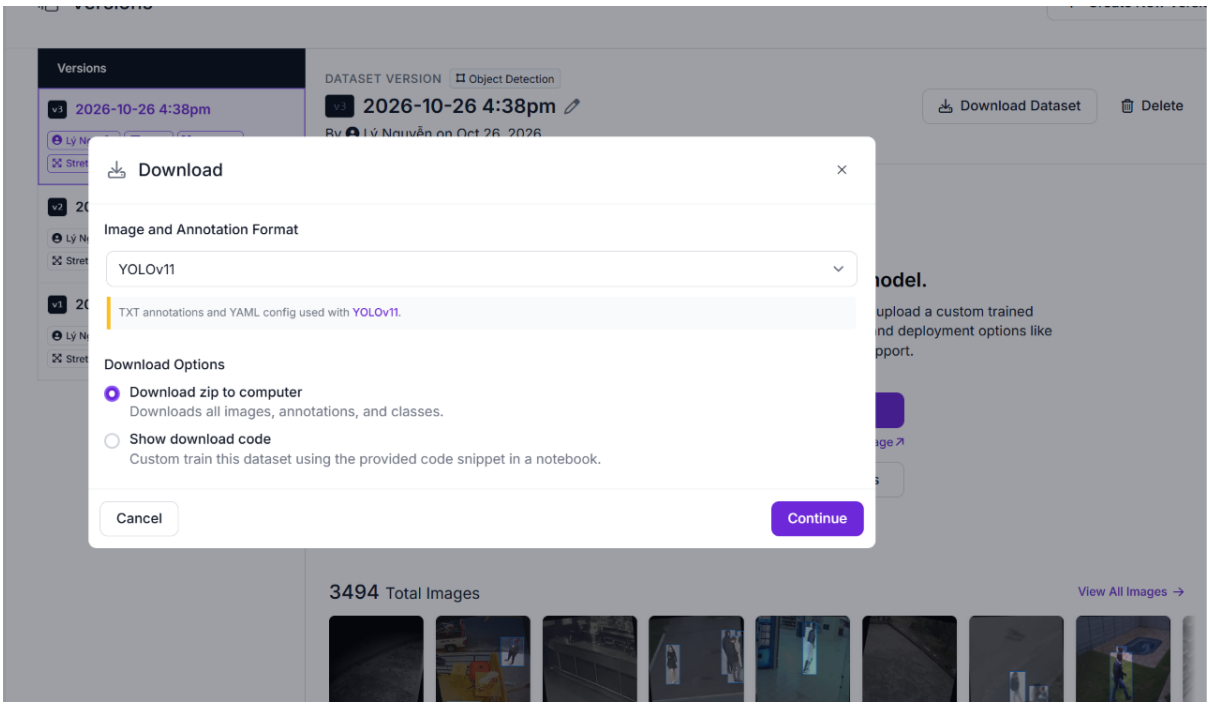



5. **Tăng cường (Augmentation):** Thêm *Horizontal Flip* (lật ngang), *Brightness* (thay đổi độ sáng), *Blur* (làm mờ). Thiết lập nhân bản dữ liệu (Generate 3x).






6. **Xuất file (Export):** Chọn định dạng **YOLOv11** (hoàn toàn tương thích với YOLO11) và tải file .zip về máy.




## GIAI ĐOẠN 3: HUẤN LUYỆN MÔ HÌNH (TRAINING YOLO11m)

Quá trình này sẽ biến máy tính Windows của bạn thành cỗ máy dạy AI.

1. **Chuẩn bị cấu trúc thư mục:** Giải nén file tải từ Roboflow, tạo thư mục dự án Training_YOLO:

Training_YOLO/

├── dataset/

│ ├── train/ (chứa thư mục images và labels)

│ ├── valid/ (chứa thư mục images và labels)

│ └── data.yaml

├── **train_model.py # Code dùng để huấn luyện AI**

└── **test_model.py # Code dùng để test AI sau khi train xong**

**Cài đặt thư viện:** Mở Terminal (hoặc CMD) gõ các lệnh để tải các thư viện cần thiết opencv, yolo, **Cài đặt PyTorch (Bản hỗ trợ GPU/CUDA):**

Bash:

pip install ultralytics

**Chỉnh sửa File data.yaml ( ĐIỂM CẦN SỬA SỐ 1)**

Khi mang sang máy khác, lỗi phổ biến nhất là Dataset not found. Lý do là đường dẫn path lưu theo máy cũ. Mở file dataset/data.yaml và sửa lại thành **đường dẫn tuyệt đối của máy tính đang chạy**.

YAML

# SỬA DÒNG NÀY: Trỏ đúng vào thư mục dataset trên máy của người đang chạy

path: D:/Thư_mục_của_bạn/Human_Counting_Pro/dataset

train: train/images val: valid/images test: test/images

nc: 1

names: ['person']

**Code Huấn luyện train_model.py ( ĐIỂM CẦN SỬA SỐ 2)**

Tạo file train_model.py. Các thành viên có cấu hình máy yếu hơn (ít RAM, ít VRAM) cần chú ý sửa tham số batch và device.

Python

from ultralytics import YOLO import multiprocessing

def main():

# Khởi tạo mô hình YOLO11 Medium

model = YOLO('yolo11m.pt') // tùy máy có thể chạy bản khác yolo11m

print("Bắt đầu huấn luyện...")

model.train( data='dataset/data.yaml', epochs=50, patience=30, imgsz=640,

# CHÚ Ý SỬA THÔNG SỐ RAM/VRAM:

# - Máy có Card RTX 4060 (8GB VRAM) / RAM 16GB: Để batch=16

# - Máy có Card yếu hơn (4GB VRAM): Hạ xuống batch=8 hoặc batch=4 để không bị lỗi Out of Memory (OOM)

batch=16,

# CHÚ Ý SỬA THÔNG SỐ THIẾT BỊ:

# - device=0: Chạy bằng Card rời NVIDIA

# - device='cpu': Chạy bằng CPU (Dành cho máy không có Card NVIDIA, nhưng sẽ rất chậm)

device=0,

# CHÚ Ý SỬA THÔNG SỐ WORKERS:

# Nếu máy hay bị treo lúc mới chạy, thêm tham số: workers=2 (hoặc workers=4) project='runs/detect',

name='yolo11m_v1' )

if __name__ == '__main__': multiprocessing.freeze_support() main()

**Code Test Mô hình test_model.py**

Sau khi quá trình huấn luyện hoàn tất, mô hình tốt nhất sẽ nằm ở đường dẫn runs/detect/yolo11m_v1/weights/best.pt. Tạo file test_model.py để nhóm kiểm tra thử thành quả trên bất kỳ ảnh hoặc video nào.

Python

from ultralytics import YOLO import cv2

# 1. Trỏ đến file trọng số (bộ não) vừa train xong

# Đảm bảo đường dẫn này đúng với máy của từng người

model = YOLO('runs/detect/yolo11m_v1/weights/best.pt')

# 2. Thay source bằng file ảnh/video muốn test, hoặc để source=0 để bật Webcam source_test = 'dataset/test/images/anh_test_bat_ky.jpg'

# source_test = 'video_test.mp4'

# source_test = 0 # Nếu muốn test Webcam trực tiếp

print(f"Đang chạy kiểm tra trên: {source_test}") results = model.predict(

source=source_test,

show=True, # Hiển thị cửa sổ kết quả

conf=0.4, # Chỉ hiển thị khung khi độ tự tin >= 40%

save=True # Lưu lại kết quả vào thư mục runs/detect/predict )

# Giữ cửa sổ hiển thị (nếu test ảnh) cv2.waitKey(0) cv2.destroyAllWindows()

**Code Test Hàng Loạt Nhiều Ảnh (Batch Inference)**

Đối với các thành viên QA/Tester muốn kiểm tra độ chính xác của mô hình trên hàng chục hoặc hàng trăm bức ảnh cùng lúc thay vì test lắt nhắt từng tấm, hãy sử dụng kịch bản test hàng loạt (Batch Testing).

**Cách thực hiện:**

1. Tạo một thư mục chứa tất cả các ảnh cần test (ví dụ: tạo thư mục test_images ở thư

mục gốc và chép ảnh vào đó, hoặc dùng luôn thư mục dataset/test/images/).

2. Tạo file test_batch.py và dán đoạn code sau: Python

from ultralytics import YOLO

def main():

# 1. Trỏ đến file mô hình đã train xong

model = YOLO('runs/detect/yolo11m_v1/weights/best.pt')

# 2. Trỏ source vào THƯ MỤC chứa ảnh thay vì 1 file ảnh cụ thể

thuc_muc_test = 'dataset/test/images' # Đường dẫn đến thư mục chứa nhiều ảnh

print(f"Đang tiến hành nhận diện hàng loạt toàn bộ ảnh trong: {thuc_muc_test}")

# 3. Chạy dự đoán

results = model.predict( source=thuc_muc_test,

conf=0.4, # Ngưỡng tự tin >= 40%

save=True, # BẮT BUỘC BẬT: Máy sẽ tự động lưu các ảnh đã vẽ khung

show=False, # TẮT: Để tránh việc máy tính bật lên hàng trăm cửa sổ ảnh cùng lúc gây treo máy

stream=True # Tối ưu hóa bộ nhớ RAM khi chạy hàng nghìn ảnh

)

# Kích hoạt vòng lặp stream (Bắt buộc phải có vòng lặp for này để máy thực sự chạy) for r in results:

pass

print("\n Hoàn tất! Hãy mở thư mục [runs/detect/predict] (hoặc predict2, predict3...) để xem toàn bộ ảnh kết quả.")

if __name__ == '__main__': main()

**Lưu ý quan trọng cho nhóm Test:**

• Khi chạy code này, màn hình sẽ không pop-up lên bức ảnh nào cả để tiết kiệm tài

nguyên. Máy sẽ chạy ngầm bằng GPU.

• Sau khi Terminal báo Hoàn tất!, các thành viên chỉ cần mở thư mục

runs/detect/predict (YOLO sẽ tự động tạo thư mục này). Bên trong sẽ là toàn bộ ảnh

đã được AI tự động vẽ khung bounding box để nhóm có thể lướt xem và đánh giá nhanh chóng

## GIAI ĐOẠN 4: LẬP TRÌNH PHẦN MỀM DESKTOP (MVC & VECTOR LINE)

Đây là giai đoạn đóng gói AI vào một giao diện chuyên nghiệp có thể click chuột sử dụng dễ dàng. Cấu trúc dự án phần mềm gồm 3 phần chính:

**1. Lõi Toán Học & AI (core/yolo_engine.py):**

- Chịu trách nhiệm load file best.pt.
- Kích hoạt thuật toán Tracking botsort.yaml (với thông số iou=0.3 khắt khe để chống gộp nhầm 2 người đi sát nhau thành 1 ID).
- **Toán học Vector:** Xử lý tọa độ click chuột của người dùng tạo thành vạch thẳng (Vector AB). Dùng *Tích có hướng (Cross Product)* để tính toán góc đâm của người cắt qua vạch. Nếu kết quả ra số Âm là đi VÀO, số Dương là đi RA (hoặc ngược lại tùy cài đặt đảo chiều).
- Quản lý Cooldown: Lưu bộ nhớ đệm 15 frames để 1 người đi loang quanh vạch không bị cộng dồn nhiều lần.

**Code mẫu:**

```
import cv2
import threading
import time
import os
from collections import deque
from ultralytics import YOLO

class YOLOEngine:
    def __init__(self, model_path, cb_stats, cb_log, cb_stop,
cb_error):
        self.model_path = model_path
        self.model = None
        self.load_error = None
        self._init_model(model_path)

        self.video_path = None
        self.cap = None
        self.is_running = False
        self.is_paused = False

```

```
        # Biến chia sẻ frame an toàn cho UI Polling (Chống đơ Tkinter 100%)
        self.latest_frame = None
        self.current_fps = 0.0

        # Cấu hình thông số AI
        self.conf_thresh = 0.40
        self.point_mode = "foot"  # "foot": Gót chân | "center": Tâm người
        self.invert_dir = False

        self.FRAME_W = 960
        self.FRAME_H = 540
        self.line_points = []

        self.track_trails = {}
        self.cooldown_ids = {}

        self.count_in = 0
        self.count_out = 0

        # Callbacks
        self.cb_stats = cb_stats
        self.cb_log = cb_log
        self.cb_stop = cb_stop
        self.cb_error = cb_error

    def _init_model(self, path):
        try:
            if not os.path.exists(path) and not path.startswith("yolo"):
                raise FileNotFoundError(f"Không tìm thấy file model tại: {path}")
            self.model = YOLO(path)
            self.model_path = path
            self.load_error = None
        except Exception as e:
            self.load_error = str(e)
            print(f"[LỖI NẠP MODEL]: {e}")

    def load_new_model(self, new_path):
        self._init_model(new_path)
        if self.load_error and self.cb_error:
            self.cb_error(self.load_error)

    def set_line(self, pt1, pt2):
        self.line_points = [list(pt1), list(pt2)]

```

```
    def reverse_line_direction(self):
        if len(self.line_points) == 2:
            self.line_points = [self.line_points[1],
self.line_points[0]]
        else:
            self.invert_dir = not self.invert_dir

    def _intersect(self, A, B, C, D):
        def ccw(p1, p2, p3):
            return (p3[1] - p1[1]) * (p2[0] - p1[0]) > (p2[1] - p1[1])
* (p3[0] - p1[0])
        return ccw(A, C, D) != ccw(B, C, D) and ccw(A, B, C) != ccw(A,
B, D)

    def draw_line_with_direction(self, frame):
        if len(self.line_points) != 2:
            return frame
        A, B = tuple(self.line_points[0]), tuple(self.line_points[1])

        cv2.line(frame, A, B, (0, 255, 255), 3, cv2.LINE_AA)
        cv2.circle(frame, A, 6, (0, 0, 255), -1, cv2.LINE_AA)
        cv2.circle(frame, B, 6, (0, 0, 255), -1, cv2.LINE_AA)

        mx, my = int((A[0] + B[0]) / 2), int((A[1] + B[1]) / 2)
        dx, dy = B[0] - A[0], B[1] - A[1]
        length = (dx**2 + dy**2) ** 0.5
        if length > 0:
            nx, ny = -dy / length, dx / length
            if self.invert_dir:
                nx, ny = -nx, -ny
            tip_x, tip_y = int(mx + nx * 45), int(my + ny * 45)
            cv2.arrowedLine(frame, (mx, my), (tip_x, tip_y), (0, 255,
0), 3, cv2.LINE_AA, tipLength=0.35)
            cv2.putText(frame, "HUONG VAO", (tip_x - 45, tip_y - 10),
cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2, cv2.LINE_AA)

        return frame

    def start(self, video_source):
        if self.model is None:
            if self.cb_error:
                self.cb_error(f"Chưa nạp được Model!\nChi tiết:
{self.load_error}")
            self.cb_stop()
            return

        self.video_path = video_source
        self.is_running = True
```

```
            self.is_paused = False
            self.count_in = 0
            self.count_out = 0
            self.track_trails.clear()
            self.cooldown_ids.clear()
            threading.Thread(target=self._process_video,
    daemon=True).start()

        def toggle_pause(self):
            self.is_paused = not self.is_paused
            return self.is_paused

        def stop(self):
            self.is_running = False
            self.is_paused = False

        def _process_video(self):
            try:
                self.cap = cv2.VideoCapture(self.video_path)
                if not self.cap.isOpened():
                    if self.cb_error:
                        self.cb_error("Không thể mở file Video này!")
                    self.is_running = False
                    self.cb_stop()
                    return

                prev_time = time.time()
                tracker_type = "botsort.yaml"

                while self.cap.isOpened() and self.is_running:
                    if self.is_paused:
                        time.sleep(0.05)
                        prev_time = time.time()
                        continue

                    ret, frame = self.cap.read()
                    if not ret:
                        break

                    frame = cv2.resize(frame, (self.FRAME_W, self.FRAME_H))

                    for tid in list(self.cooldown_ids.keys()):
                        self.cooldown_ids[tid] -= 1
                        if self.cooldown_ids[tid] <= 0:
                            del self.cooldown_ids[tid]

                    self.draw_line_with_direction(frame)

```

```
                # Chạy Tracking (Tự động fallback nếu máy thiếu thư
viện của botsort)
                try:
                    results = self.model.track(
                        frame, persist=True, tracker=tracker_type,
                        conf=self.conf_thresh, verbose=False
                    )
                except Exception:
                    tracker_type = "bytetrack.yaml"
                    results = self.model.track(
                        frame, persist=True, tracker=tracker_type,
                        conf=self.conf_thresh, verbose=False
                    )

                if results and results[0].boxes is not None and
results[0].boxes.id is not None:
                    boxes = results[0].boxes.xyxy.cpu().numpy()
                    track_ids = results[0].boxes.id.int().cpu().numpy()
                    confs = results[0].boxes.conf.cpu().numpy()

                    for box, track_id, conf in zip(boxes, track_ids,
confs):
                        x1, y1, x2, y2 = map(int, box)
                        cx = (x1 + x2) // 2
                        cy = y2 if self.point_mode == "foot" else (y1 +
y2) // 2
                        current_pt = (cx, cy)

                        if track_id not in self.track_trails:
                            self.track_trails[track_id] =
deque(maxlen=15)

                        if len(self.track_trails[track_id]) > 0 and
len(self.line_points) == 2:
                            prev_pt = self.track_trails[track_id][-1]
                            A, B = self.line_points[0],
self.line_points[1]

                            if track_id not in self.cooldown_ids and
self._intersect(A, B, prev_pt, current_pt):
                                vAB = (B[0] - A[0], B[1] - A[1])
                                vP = (current_pt[0] - prev_pt[0],
current_pt[1] - prev_pt[1])
                                cross_product = vAB[0] * vP[1] - vAB[1]
* vP[0]

                                if self.invert_dir:
                                    cross_product = -cross_product
```

```
                                if cross_product > 0:
                                    self.count_in += 1
                                    self.cooldown_ids[track_id] = 15
                                    self.cb_stats(self.count_in,
self.count_out, 1, 0)
                                    self.cb_log(track_id, "Vào cổng",
round(float(conf), 2))
                                    cv2.line(frame, tuple(A), tuple(B),
(0, 255, 0), 6, cv2.LINE_AA)
                                elif cross_product < 0:
                                    self.count_out += 1
                                    self.cooldown_ids[track_id] = 15
                                    self.cb_stats(self.count_in,
self.count_out, 0, 1)
                                    self.cb_log(track_id, "Ra cổng",
round(float(conf), 2))
                                    cv2.line(frame, tuple(A), tuple(B),
(0, 0, 255), 6, cv2.LINE_AA)

                        self.track_trails[track_id].append(current_pt)

                        pts = list(self.track_trails[track_id])
                        for i in range(1, len(pts)):
                            cv2.line(frame, pts[i - 1], pts[i], (0,
255, 180), 2, cv2.LINE_AA)

                        color = (25, 135, 84) if track_id not in
self.cooldown_ids else (0, 215, 255)
                        cv2.rectangle(frame, (x1, y1), (x2, y2), color,
2)
                        cv2.circle(frame, current_pt, 5, (0, 0, 255), -
1)

                        label = f"ID {track_id}"
                        (tw, th), _ = cv2.getTextSize(label,
cv2.FONT_HERSHEY_SIMPLEX, 0.55, 2)
                        cv2.rectangle(frame, (x1, y1 - th - 8), (x1 +
tw + 6, y1), color, -1)
                        cv2.putText(frame, label, (x1 + 3, y1 - 4),
cv2.FONT_HERSHEY_SIMPLEX, 0.55, (255, 255, 255), 2, cv2.LINE_AA)

                curr_time = time.time()
                dt = curr_time - prev_time
                self.current_fps = (1.0 / dt) if dt > 0 else 0.0
                prev_time = curr_time

                # Cập nhật frame mới nhất cho UI Polling lấy
```

```
                self.latest_frame = frame
                time.sleep(0.005) # Nhường nhịp CPU cho Giao diện Tkinter mượt mà

        except Exception as e:
            if self.cb_error:
                self.cb_error(f"Lỗi trong lúc chạy Video:\n{str(e)}")
        finally:
            if self.cap:
                self.cap.release()
            self.is_running = False
            self.cb_stop()
```

**2. Giao diện Người dùng (ui/dashboard_pro.py):**

- Dùng thư viện CustomTkinter để thiết kế giao diện phẳng (Light Mode) gồm thanh Sidebar menu, các thẻ Card hiển thị số tổng, và một tk.Canvas cực nhạy để vẽ vạch.
- Kết hợp matplotlib để vẽ trực tiếp biểu đồ cột theo ngày lên phần mềm.
- Dùng pandas để theo dõi và tự động lưu/cập nhật dữ liệu lượt đi Vào/Ra vào file thong_ke_ngay.csv nhằm theo dõi dài hạn.

```
import os
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import customtkinter as ctk
import cv2
from PIL import Image, ImageTk
import datetime
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

ctk.set_appearance_mode("Light")
BG_COLOR = "#F0F4F8"
SIDEBAR_COLOR = "#0D2546"
CARD_COLOR = "#FFFFFF"
TEXT_MAIN = "#2B3445"
GREEN_ACCENT = "#198754"
BLUE_ACCENT = "#0D6EFD"
RED_ACCENT = "#DC3545"
ORANGE_ACCENT = "#FD7E14"

DAILY_CSV_FILE = "thong_ke_ngay.csv"

class ProDashboard(ctk.CTkFrame):
    def __init__(self, master, engine):
        super().__init__(master, fg_color=BG_COLOR)
```

```
        self.engine = engine
        self.preview_frame = None
        self.current_display_frame = None
        self.click_points = []
        self.log_data = []

        self.img_scale = 1.0
        self.img_offset_x = 0
        self.img_offset_y = 0

        self.init_daily_csv()
        self.setup_ui()
        self.refresh_daily_summary_ui()
        self.update_clock()

    # ================= QUẢN LÝ FILE CSV THỐNG KÊ THEO NGÀY =================
    def init_daily_csv(self):
        if not os.path.exists(DAILY_CSV_FILE):
            df = pd.DataFrame(columns=["Ngày", "Tổng Vào", "Tổng Ra", "Cập nhật cuối"])
            df.to_csv(DAILY_CSV_FILE, index=False, encoding="utf-8-sig")

    def update_daily_csv(self, add_in=0, add_out=0):
        today_str = datetime.datetime.now().strftime("%d/%m/%Y")
        now_time = datetime.datetime.now().strftime("%H:%M:%S")

        try:
            df = pd.read_csv(DAILY_CSV_FILE, encoding="utf-8-sig")
        except Exception:
            df = pd.DataFrame(columns=["Ngày", "Tổng Vào", "Tổng Ra", "Cập nhật cuối"])

        if today_str in df["Ngày"].values:
            idx = df.index[df["Ngày"] == today_str][0]
            df.loc[idx, "Tổng Vào"] = int(df.loc[idx, "Tổng Vào"]) + add_in
            df.loc[idx, "Tổng Ra"] = int(df.loc[idx, "Tổng Ra"]) + add_out
            df.loc[idx, "Cập nhật cuối"] = now_time
        else:
            new_row = pd.DataFrame([{
                "Ngày": today_str,
                "Tổng Vào": add_in,
                "Tổng Ra": add_out,
                "Cập nhật cuối": now_time
            }])
```

```
            df = pd.concat([df, new_row], ignore_index=True)

        df.to_csv(DAILY_CSV_FILE, index=False, encoding="utf-8-sig")
        self.refresh_daily_summary_ui()

    def setup_ui(self):
        self.grid_columnconfigure(0, weight=0)
        self.grid_columnconfigure(1, weight=5)
        self.grid_columnconfigure(2, weight=2)
        self.grid_rowconfigure(0, weight=0)
        self.grid_rowconfigure(1, weight=6)
        self.grid_rowconfigure(2, weight=3)

        self.setup_sidebar()
        self.setup_header()
        self.setup_video_section()
        self.setup_kpi_section()
        self.setup_daily_csv_section() # Thay cho biểu đồ theo giờ cũ
        self.setup_table_section()

    # ================ 1. SIDEBAR ================
    def setup_sidebar(self):
        sidebar = ctk.CTkFrame(self, width=220, fg_color=SIDEBAR_COLOR,
corner_radius=0)
        sidebar.grid(row=0, column=0, rowspan=3, sticky="nsew")
        sidebar.grid_propagate(False)

        ctk.CTkLabel(sidebar, text="TDV", font=("Arial", 19, "bold"),
text_color="white", justify="left").pack(pady=(25, 30), padx=20,
anchor="w")

        self.nav_btns = {}
        menu_items = [
            ("overview", "🏠 Tổng quan", lambda: None),
            ("camera", "📹 Nguồn Video", self.select_video),
            ("webcam", "📷 Webcam Live", self.open_webcam),
            ("history", "🕒 Chi tiết", self.open_full_history_modal),
            ("settings", "⚙️ Cài đặt AI & Model",
self.open_settings_modal)
        ]

        for key, text, cmd in menu_items:
            btn = ctk.CTkButton(
                sidebar, text=text, fg_color=BLUE_ACCENT if key ==
"overview" else "transparent",
                text_color="white", hover_color="#1B3A68",
font=("Arial", 13, "bold"),
                anchor="w", height=42, corner_radius=8,
```

```
                    command=lambda k=key, c=cmd: self.activate_nav(k, c)
                )
                btn.pack(pady=4, padx=15, fill="x")
                self.nav_btns[key] = btn

            ctk.CTkLabel(
                sidebar, text="DEV:Lý-Linh-Nguyên-Lộc\nPhiên bản Pro",
                text_color="#A0ABC0", justify="left", font=("Arial", 11)
            ).pack(side="bottom", pady=20, padx=20, anchor="w")

        def activate_nav(self, key, callback):
            for k, b in self.nav_btns.items():
                b.configure(fg_color=BLUE_ACCENT if k == key else
"transparent")
            callback()

        # ================= 2. HEADER =================
        def setup_header(self):
            header = ctk.CTkFrame(self, fg_color=BG_COLOR, corner_radius=0,
height=55)
            header.grid(row=0, column=1, columnspan=2, sticky="ew",
padx=20, pady=(10, 0))

            ctk.CTkLabel(header, text="👥 HỆ THỐNG ĐẾM NGƯỜI VÀO - RA
CỔNG", font=("Arial", 21, "bold"),
text_color=TEXT_MAIN).pack(side="left", pady=5)
            self.lbl_clock = ctk.CTkLabel(header, text="", font=("Arial",
13, "bold"), text_color="#555555", justify="right")
            self.lbl_clock.pack(side="right", pady=5)

        # ================= 3. KHU VỰC VIDEO =================
        def setup_video_section(self):
            video_card = ctk.CTkFrame(self, fg_color=CARD_COLOR,
corner_radius=12)
            video_card.grid(row=1, column=1, sticky="nsew", padx=(20, 10),
pady=10)
            video_card.grid_rowconfigure(1, weight=1)
            video_card.grid_columnconfigure(0, weight=1)

            top_bar = ctk.CTkFrame(video_card, fg_color="transparent")
            top_bar.grid(row=0, column=0, sticky="ew", padx=15, pady=(10,
5))
            self.lbl_cam_title = ctk.CTkLabel(top_bar, text="Màn hình Giám
sát (Click 2 điểm lên hình để kẻ vạch)", font=("Arial", 13, "bold"),
text_color=TEXT_MAIN)
            self.lbl_cam_title.pack(side="left")

```

```
        self.lbl_badge = ctk.CTkLabel(top_bar, text=" YOLOv11 + Tracking ", fg_color="#E2E8F0", text_color="#334155", corner_radius=6, font=("Arial", 11, "bold"))
        self.lbl_badge.pack(side="right", padx=5)

        self.video_canvas = tk.Canvas(video_card, bg="#181C24", highlightthickness=0, cursor="crosshair")
        self.video_canvas.grid(row=1, column=0, sticky="nsew", padx=15, pady=5)
        self.video_canvas.bind("<Button-1>", self.on_canvas_click)
        self.video_canvas.bind("<Configure>", self.on_canvas_resize)

        # Tạo text hướng dẫn và tự động neo vào tâm Canvas
        self.canvas_text = self.video_canvas.create_text(
            0, 0, text="1. Nhấn '🗀 Chọn video'\n2. Click 2 điểm kẻ vạch\n3. Nhấn '▶ Bắt đầu'",
            fill="#CBD5E1", font=("Arial", 13, "bold"), justify="center", anchor="center"
        )
        # Tự động dời chữ về giữa mỗi khi khung Canvas thay đổi kích thước
        self.video_canvas.bind("<Configure>", lambda e: self.video_canvas.coords(self.canvas_text, e.width // 2, e.height // 2))

        ctrl_bar = ctk.CTkFrame(video_card, fg_color="transparent")
        ctrl_bar.grid(row=2, column=0, sticky="ew", padx=15, pady=(5, 12))

        btn_cfg = {"height": 34, "corner_radius": 6, "font": ("Arial", 12, "bold")}

        self.btn_select = ctk.CTkButton(ctrl_bar, text="🗀 Chọn video", fg_color="#F1F5F9", text_color=TEXT_MAIN, hover_color="#E2E8F0", width=105, command=self.select_video, **btn_cfg)
        self.btn_select.pack(side="left", padx=4)

        self.btn_play = ctk.CTkButton(ctrl_bar, text="▶ Bắt đầu", fg_color=BLUE_ACCENT, hover_color="#0B5ED7", width=95, command=self.toggle_tracking, **btn_cfg)
        self.btn_play.pack(side="left", padx=4)

        self.btn_pause = ctk.CTkButton(ctrl_bar, text="⏸ Tạm dừng", fg_color="#F1F5F9", text_color=TEXT_MAIN, hover_color="#E2E8F0", width=100, state="disabled", command=self.pause_tracking, **btn_cfg)
        self.btn_pause.pack(side="left", padx=4)

```

```
        self.btn_redraw = ctk.CTkButton(ctrl_bar, text="🔄 Kẻ lại vạch", fg_color="#F1F5F9", text_color=TEXT_MAIN, hover_color="#E2E8F0", width=100, command=self.reset_line, **btn_cfg)
        self.btn_redraw.pack(side="left", padx=4)

        self.btn_flip = ctk.CTkButton(ctrl_bar, text="↕️ Đảo hướng Vào/Ra", fg_color="#F1F5F9", text_color=TEXT_MAIN, hover_color="#E2E8F0", width=130, command=self.flip_direction, **btn_cfg)
        self.btn_flip.pack(side="left", padx=4)

        self.lbl_status = ctk.CTkLabel(ctrl_bar, text="FPS: 0.0 | Conf: 0.40", font=("Arial", 11, "bold"), text_color="#64748B")
        self.lbl_status.pack(side="right", padx=5)

    # ================ 4. KHU VỰC KPI (PHẢI) ================
    def setup_kpi_section(self):
        kpi_frame = ctk.CTkFrame(self, fg_color="transparent")
        kpi_frame.grid(row=1, column=2, sticky="nsew", padx=(10, 20), pady=10)
        kpi_frame.grid_rowconfigure((0, 1, 2), weight=1)
        kpi_frame.grid_columnconfigure(0, weight=1)

        card_in = ctk.CTkFrame(kpi_frame, fg_color=CARD_COLOR, corner_radius=12)
        card_in.grid(row=0, column=0, sticky="nsew", pady=(0, 8))
        ctk.CTkLabel(card_in, text="🚶 Lượt vào hiện tại ", font=("Arial", 14, "bold"), text_color=TEXT_MAIN).pack(pady=(14, 0))
        self.lbl_in = ctk.CTkLabel(card_in, text="0", font=("Arial", 48, "bold"), text_color=GREEN_ACCENT)
        self.lbl_in.pack(pady=2)

        card_out = ctk.CTkFrame(kpi_frame, fg_color=CARD_COLOR, corner_radius=12)
        card_out.grid(row=1, column=0, sticky="nsew", pady=8)
        ctk.CTkLabel(card_out, text="🚶 Lượt ra hiện tại", font=("Arial", 14, "bold"), text_color=TEXT_MAIN).pack(pady=(12, 0))
        self.lbl_out = ctk.CTkLabel(card_out, text="0", font=("Arial", 44, "bold"), text_color=RED_ACCENT)
        self.lbl_out.pack(pady=0)
        self.lbl_inside = ctk.CTkLabel(card_out, text="Đang ở bên trong: 0 người", font=("Arial", 12, "bold"), text_color=BLUE_ACCENT)
        self.lbl_inside.pack(pady=(0, 8))

        card_cam = ctk.CTkFrame(kpi_frame, fg_color=CARD_COLOR, corner_radius=12)
        card_cam.grid(row=2, column=0, sticky="nsew", pady=(8, 0))
```

```
        ctk.CTkLabel(card_cam, text="📡 Trạng thái hệ thống",
font=("Arial", 14, "bold"), text_color=TEXT_MAIN).pack(pady=(14, 6))
        self.lbl_cam_state = ctk.CTkLabel(
            card_cam, text="◯ Chưa chọn video", font=("Arial", 13,
"bold"),
            text_color="#475569", fg_color="#F1F5F9", corner_radius=6,
padx=14, pady=6
        )
        self.lbl_cam_state.pack()

    # ================= 5. THỐNG KÊ TỔNG HỢP THEO NGÀY (LƯU CSV)
================
    def setup_daily_csv_section(self):
        daily_card = ctk.CTkFrame(self, fg_color=CARD_COLOR,
corner_radius=12)
        daily_card.grid(row=2, column=1, sticky="nsew", padx=(20, 10),
pady=(5, 15))
        daily_card.grid_columnconfigure(0, weight=3) # Biểu đồ cột theo
ngày
        daily_card.grid_columnconfigure(1, weight=2) # Bảng số liệu CSV
theo ngày
        daily_card.grid_rowconfigure(1, weight=1)

        top = ctk.CTkFrame(daily_card, fg_color="transparent")
        top.grid(row=0, column=0, columnspan=2, sticky="ew", padx=15,
pady=(10, 2))
        ctk.CTkLabel(
            top, text="🧮 Thống kê Tổng Vào / Ra theo Ngày (Tự động
lưu)",
            font=("Arial", 14, "bold"), text_color=TEXT_MAIN
        ).pack(side="left")

        ctk.CTkButton(
            top, text="🗑 Xóa dữ liệu ngày", fg_color="#F1F5F9",
text_color=RED_ACCENT, hover_color="#FEE2E2",
            height=26, width=110, font=("Arial", 11, "bold"),
command=self.clear_daily_csv
        ).pack(side="right", padx=4)

        ctk.CTkButton(
            top, text="📤 Xuất CSV ra chỗ khác", fg_color="#E8F5E9",
text_color=GREEN_ACCENT, hover_color="#C8E6C9",
            height=26, width=135, font=("Arial", 11, "bold"),
command=self.export_daily_csv_copy
        ).pack(side="right", padx=4)

        # Trái: Biểu đồ Cột theo Ngày
        self.fig, self.ax = plt.subplots(figsize=(4.2, 1.9), dpi=100)
```

```
        self.fig.patch.set_facecolor('#FFFFFF')
        self.chart_canvas = FigureCanvasTkAgg(self.fig,
master=daily_card)
        self.chart_canvas.get_tk_widget().grid(row=1, column=0,
sticky="nsew", padx=(10, 5), pady=5)

        # Phải: Bảng CSV theo Ngày
        cols = ("Date", "In", "Out", "Updated")
        self.tree_daily = ttk.Treeview(daily_card, columns=cols,
show="headings", height=4)
        self.tree_daily.heading("Date", text="Ngày");
self.tree_daily.column("Date", width=85, anchor="center")
        self.tree_daily.heading("In", text="Tổng Vào");
self.tree_daily.column("In", width=65, anchor="center")
        self.tree_daily.heading("Out", text="Tổng Ra");
self.tree_daily.column("Out", width=65, anchor="center")
        self.tree_daily.heading("Updated", text="Cập nhật");
self.tree_daily.column("Updated", width=75, anchor="center")
        self.tree_daily.grid(row=1, column=1, sticky="nsew", padx=(5,
15), pady=(5, 10))

    def refresh_daily_summary_ui(self):
        try:
            df = pd.read_csv(DAILY_CSV_FILE, encoding="utf-8-sig")
        except Exception:
            return

        # Cập nhật bảng Treeview
        for item in self.tree_daily.get_children():
            self.tree_daily.delete(item)
        for _, row in df.iloc[::-1].iterrows():
            self.tree_daily.insert("", "end", values=(row["Ngày"],
int(row["Tổng Vào"]), int(row["Tổng Ra"]), row["Cập nhật cuối"]))

        # Cập nhật biểu đồ cột (Lấy tối đa 7 ngày gần nhất)
        df_recent = df.tail(7)
        dates = df_recent["Ngày"].tolist() if not df_recent.empty else
[datetime.datetime.now().strftime("%d/%m/%Y")]
        in_vals = df_recent["Tổng Vào"].astype(int).tolist() if not
df_recent.empty else [0]
        out_vals = df_recent["Tổng Ra"].astype(int).tolist() if not
df_recent.empty else [0]

        self.ax.clear()
        self.ax.set_facecolor('#FFFFFF')
        x = range(len(dates))
        w = 0.32
```

```
        bars1 = self.ax.bar([i - w/2 for i in x], in_vals, width=w, color='#28C76F', label='Tổng Vào')
        bars2 = self.ax.bar([i + w/2 for i in x], out_vals, width=w, color='#EA5455', label='Tổng Ra')

        self.ax.set_xticks(list(x))
        self.ax.set_xticklabels(dates, fontsize=8, color='#333333', fontweight='bold')
        self.ax.tick_params(axis='y', labelsize=8, colors='#555555')
        self.ax.spines['top'].set_visible(False)
        self.ax.spines['right'].set_visible(False)
        self.ax.spines['left'].set_color('#E2E8F0')
        self.ax.spines['bottom'].set_color('#E2E8F0')
        self.ax.set_ylim(0, max(max(in_vals + out_vals, default=5) + 4, 5))
        self.ax.legend(frameon=False, fontsize=8, loc='upper right')

        for bar in bars1:
            h = bar.get_height()
            if h > 0:
                self.ax.text(bar.get_x() + bar.get_width()/2, h + 0.1, int(h), ha='center', va='bottom', fontsize=8, color='#198754', fontweight='bold')
        for bar in bars2:
            h = bar.get_height()
            if h > 0:
                self.ax.text(bar.get_x() + bar.get_width()/2, h + 0.1, int(h), ha='center', va='bottom', fontsize=8, color='#DC3545', fontweight='bold')

        self.fig.tight_layout()
        self.chart_canvas.draw_idle()

    def export_daily_csv_copy(self):
        save_path = filedialog.asksaveasfilename(
            defaultextension=".csv",
            initialfile=f"ThongKe_TheoNgay_{datetime.datetime.now().strftime('%Y%m%d')}.csv",
            filetypes=[("CSV File", "*.csv"), ("Excel File", "*.xlsx")]
        )
        if save_path:
            df = pd.read_csv(DAILY_CSV_FILE, encoding="utf-8-sig")
            if save_path.endswith(".xlsx"):
                df.to_excel(save_path, index=False)
            else:
                df.to_csv(save_path, index=False, encoding="utf-8-sig")
            messagebox.showinfo("Thành công", f"Đã xuất bảng tổng hợp theo ngày tại:\n{save_path}")
```

```
•
•       def clear_daily_csv(self):
•           if messagebox.askyesno("Xác nhận", "Bạn có chắc muốn xóa làm
    mới lịch sử thống kê các ngày trong CSV?"):
•               df = pd.DataFrame(columns=["Ngày", "Tổng Vào", "Tổng Ra",
    "Cập nhật cuối"])
•               df.to_csv(DAILY_CSV_FILE, index=False, encoding="utf-8-
    sig")
•               self.refresh_daily_summary_ui()
•
•       # ================= 6. BẢNG LỊCH SỬ TỪNG LƯỢT (PHẢI DƯỚI)
    =================
•       def setup_table_section(self):
•           table_card = ctk.CTkFrame(self, fg_color=CARD_COLOR,
    corner_radius=12)
•           table_card.grid(row=2, column=2, sticky="nsew", padx=(10, 20),
    pady=(5, 15))
•
•           top_bar = ctk.CTkFrame(table_card, fg_color="transparent")
•           top_bar.pack(fill="x", padx=15, pady=(10, 5))
•           ctk.CTkLabel(top_bar, text="🕒 Lịch sử từng người",
    font=("Arial", 14, "bold"), text_color=TEXT_MAIN).pack(side="left")
•
•           ctk.CTkButton(
•               top_bar, text="Xem tất cả ➔", fg_color="transparent",
    text_color=BLUE_ACCENT,
•               hover_color="#EFF6FF", font=("Arial", 12, "bold"),
    width=80, height=24,
•               command=self.open_full_history_modal
•           ).pack(side="right")
•
•           style = ttk.Style()
•           style.theme_use("default")
•           style.configure("Treeview", background="white",
    foreground="#2B3445", rowheight=26, fieldbackground="white",
    borderwidth=0, font=("Arial", 10))
•           style.configure("Treeview.Heading", background="#F8FAFC",
    foreground="#475569", font=('Arial', 10, 'bold'), borderwidth=0)
•
•           self.tree = ttk.Treeview(table_card, columns=("Time", "ID",
    "Event"), show="headings", height=5)
•           self.tree.heading("Time", text="Thời gian");
    self.tree.column("Time", width=75, anchor="center")
•           self.tree.heading("ID", text="Mã ID"); self.tree.column("ID",
    width=70, anchor="center")
•           self.tree.heading("Event", text="Sự kiện");
    self.tree.column("Event", width=85, anchor="center")
•           self.tree.pack(fill="both", expand=True, padx=12, pady=(0, 10))
```

```
    # ================= LOGIC HIỂN THỊ & VẼ VẠCH =================
    def update_clock(self):
        now = datetime.datetime.now()
        days = ["Chủ nhật", "Thứ 2", "Thứ 3", "Thứ 4", "Thứ 5", "Thứ
6", "Thứ 7"]
        self.lbl_clock.configure(text=f"{days[now.isoweekday()%7]},
{now.strftime('%d/%m/%Y')}  |  {now.strftime('%H:%M:%S')}")
        self.after(1000, self.update_clock)

    def on_canvas_resize(self, event):
        if self.current_display_frame is not None and not
self.engine.is_running:
            self.render_frame_on_canvas(self.current_display_frame)

    def render_frame_on_canvas(self, frame_bgr):
        self.current_display_frame = frame_bgr
        cw = self.video_canvas.winfo_width()
        ch = self.video_canvas.winfo_height()
        if cw < 50 or ch < 50:
            cw, ch = 800, 450

        fh, fw = frame_bgr.shape[:2]
        scale = min(cw / fw, ch / fh)
        new_w, new_h = int(fw * scale), int(fh * scale)

        self.img_scale = scale
        self.img_offset_x = (cw - new_w) // 2
        self.img_offset_y = (ch - new_h) // 2

        resized = cv2.resize(frame_bgr, (new_w, new_h))
        img_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
        self.tk_img = ImageTk.PhotoImage(Image.fromarray(img_rgb))

        self.video_canvas.delete("all")
        self.video_canvas.create_image(self.img_offset_x,
self.img_offset_y, anchor="nw", image=self.tk_img)

    def on_canvas_click(self, event):
        if self.engine.is_running:
            messagebox.showinfo("Đang chạy", "Hãy nhấn 'Dừng hẳn' trước
khi kẻ lại vạch mới.")
            return
        if self.preview_frame is None:
            messagebox.showwarning("Chưa có video", "Vui lòng nhấn '
Chọn video' trước khi kẻ vạch!")
            return

```

```
            if len(self.click_points) >= 2:
                self.click_points.clear()

            orig_x = int((event.x - self.img_offset_x) / self.img_scale)
            orig_y = int((event.y - self.img_offset_y) / self.img_scale)

            orig_x = max(0, min(orig_x, self.engine.FRAME_W - 1))
            orig_y = max(0, min(orig_y, self.engine.FRAME_H - 1))

            self.click_points.append([orig_x, orig_y])
            self.redraw_preview_line()

        def redraw_preview_line(self):
            if self.preview_frame is None:
                return
            temp = self.preview_frame.copy()
            if len(self.click_points) == 1:
                cv2.circle(temp, tuple(self.click_points[0]), 6, (0, 0,
    255), -1, cv2.LINE_AA)
                cv2.putText(temp, "Diem 1 (Click diem thu 2...)",
    (self.click_points[0][0] + 10, self.click_points[0][1]),
    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 255), 2)
            elif len(self.click_points) == 2:
                self.engine.set_line(self.click_points[0],
    self.click_points[1])
                self.engine.draw_line_with_direction(temp)

            self.render_frame_on_canvas(temp)

        # ================= CÁC HÀM SỰ KIỆN NÚT BẤM =================
        def select_video(self):
            if self.engine.is_running:
                self.engine.stop()
            path = filedialog.askopenfilename(filetypes=[("Video files",
    "*.mp4 *.avi *.mkv *.mov"), ("All files", "*.*")])
            if path:
                self.load_video_source(path, f"📹 Video:
    {os.path.basename(path)}")

        def open_webcam(self):
            if self.engine.is_running:
                self.engine.stop()
            self.load_video_source(0, "📷 Camera trực tiếp (Webcam 0)")

        def load_video_source(self, source, title_text):
            cap = cv2.VideoCapture(source)
            ret, frame = cap.read()
            cap.release()
```

```
            if not ret:
                messagebox.showerror("Lỗi", "Không thể đọc nguồn Video
    này!")
                return

            self.engine.video_path = source
            self.lbl_cam_title.configure(text=title_text)
            self.preview_frame = cv2.resize(frame, (self.engine.FRAME_W,
    self.engine.FRAME_H))

            if len(self.engine.line_points) != 2:
                p1 = [int(self.engine.FRAME_W * 0.2),
    int(self.engine.FRAME_H * 0.65)]
                p2 = [int(self.engine.FRAME_W * 0.8),
    int(self.engine.FRAME_H * 0.65)]
                self.click_points = [p1, p2]
                self.engine.set_line(p1, p2)

            self.redraw_preview_line()
            self.lbl_cam_state.configure(text="🟢 Đã sẵn sàng (Nhấn Bắt
    đầu)", text_color=GREEN_ACCENT, fg_color="#E8F5E9")

        def reset_line(self):
            self.click_points.clear()
            self.engine.line_points.clear()
            if self.preview_frame is not None:
                self.render_frame_on_canvas(self.preview_frame)

        def flip_direction(self):
            self.engine.reverse_line_direction()
            if len(self.engine.line_points) == 2:
                self.click_points = list(self.engine.line_points)
            if not self.engine.is_running and self.preview_frame is not
    None:
                self.redraw_preview_line()

        def toggle_tracking(self):
            if self.engine.video_path is None:
                messagebox.showwarning("Chưa chọn Video", "Vui lòng nhấn
    '📁 Chọn video' trước!")
                return
            if not self.engine.is_running:
                if len(self.engine.line_points) != 2:
                    messagebox.showwarning("Chưa kẻ vạch", "Vui lòng click
    chuột 2 điểm trên hình để kẻ vạch đếm!")
                    return
```

```
            self.btn_play.configure(text="■ Dừng hẳn",
fg_color=RED_ACCENT, hover_color="#BB2D3B")
            self.btn_pause.configure(state="normal", text="⏸ Tạm dừng",
fg_color="#F1F5F9", text_color=TEXT_MAIN)
            self.btn_select.configure(state="disabled")
            self.lbl_in.configure(text="0")
            self.lbl_out.configure(text="0")
            self.lbl_inside.configure(text="Đang ở bên trong: 0 người")
            self.lbl_cam_state.configure(text="🟢 Đang chạy
Tracking...", text_color=GREEN_ACCENT, fg_color="#E8F5E9")

            self.engine.start(self.engine.video_path)
            self.master.start_ui_polling() # Kích hoạt vòng lặp vẽ hình
chống đơ
        else:
            self.engine.stop()

    def pause_tracking(self):
        if not self.engine.is_running:
            return
        paused = self.engine.toggle_pause()
        if paused:
            self.btn_pause.configure(text="▶ Tiếp tục",
fg_color=ORANGE_ACCENT, text_color="white")
            self.lbl_cam_state.configure(text="🟠 Đang tạm dừng",
text_color=ORANGE_ACCENT, fg_color="#FFF3CD")
        else:
            self.btn_pause.configure(text="⏸ Tạm dừng",
fg_color="#F1F5F9", text_color=TEXT_MAIN)
            self.lbl_cam_state.configure(text="🟢 Đang chạy
Tracking...", text_color=GREEN_ACCENT, fg_color="#E8F5E9")

    # ================= CỬA SỔ POPUP =================
    def open_full_history_modal(self):
        win = ctk.CTkToplevel(self)
        win.title("Lịch sử chi tiết từng lượt người")
        win.geometry("620x450")
        win.attributes("-topmost", True)

        ctk.CTkLabel(win, text="🕒 CHI TIẾT TỪNG LƯỢT QUA CỔNG TRONG
PHIÊN", font=("Arial", 16, "bold")).pack(pady=12)

        cols = ("STT", "Ngày", "Thời gian", "Mã ID", "Sự kiện", "Độ tin
cậy")
        tree_all = ttk.Treeview(win, columns=cols, show="headings")
        for c in cols:
            tree_all.heading(c, text=c)
            tree_all.column(c, anchor="center", width=95)
```

```
        tree_all.pack(fill="both", expand=True, padx=15, pady=10)

        for idx, item in enumerate(reversed(self.log_data), 1):
            tree_all.insert("", "end", values=(idx, item["Ngày"],
item["Thời gian"], f"ID {item['Mã Định Danh']}", item["Hành Động"],
item["Độ tin cậy"]))

    def open_settings_modal(self):
        win = ctk.CTkToplevel(self)
        win.title("Cài đặt thông số AI & Tracking")
        win.geometry("420x360")
        win.attributes("-topmost", True)

        ctk.CTkLabel(win, text="⚙ CÀI ĐẶT HỆ THỐNG AI", font=("Arial",
16, "bold")).pack(pady=15)

        lbl_c = ctk.CTkLabel(win, text=f"Ngưỡng tin cậy (Confidence):
{self.engine.conf_thresh:.2f}", font=("Arial", 13, "bold"))
        lbl_c.pack(pady=(5, 2))

        def on_conf_change(val):
            self.engine.conf_thresh = round(float(val), 2)
            lbl_c.configure(text=f"Ngưỡng tin cậy (Confidence):
{self.engine.conf_thresh:.2f}")

        slider = ctk.CTkSlider(win, from_=0.15, to=0.85,
number_of_steps=70, command=on_conf_change)
        slider.set(self.engine.conf_thresh)
        slider.pack(pady=5, padx=30, fill="x")

        ctk.CTkLabel(win, text="Điểm xét va chạm qua vạch:",
font=("Arial", 13, "bold")).pack(pady=(15, 5))
        pt_var = ctk.StringVar(value=self.engine.point_mode)

        def change_pt():
            self.engine.point_mode = pt_var.get()

        ctk.CTkRadioButton(win, text="Gót chân (Khuyên dùng cho góc cam
nghiêng)", variable=pt_var, value="foot",
command=change_pt).pack(pady=3)
        ctk.CTkRadioButton(win, text="Tâm cơ thể (Dùng cho cam ngang
tầm mắt)", variable=pt_var, value="center",
command=change_pt).pack(pady=3)

        def pick_model():
            p = filedialog.askopenfilename(filetypes=[("YOLO Weights",
"*.pt")])
            if p:
```

```
                self.engine.load_new_model(p)
                if not self.engine.load_error:
                    messagebox.showinfo("Đã đổi Model", f"Đã nạp thành
công trọng số:\n{p}")

        ctk.CTkButton(win, text="📂 Đổi file trọng số (.pt) khác...",
fg_color=BLUE_ACCENT, command=pick_model).pack(pady=20)
```

**3. Khởi chạy Hệ thống (main.py):**

- Là cầu nối giữa core và ui. Sử dụng cơ chế Threading (chạy nền luồng AI) và cơ chế Polling self.after(25) của Tkinter để liên tục lấy ảnh từ AI dán lên giao diện mà không làm treo hay giật ứng dụng.

```
import os
import customtkinter as ctk
from tkinter import messagebox
import datetime
from core.yolo_engine import YOLOEngine
from ui.dashboard_pro import ProDashboard

class App(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.title("Hệ Thống Đếm Người Vào Ra Cổng - YOLOv11 & BoT-SORT
Pro")
        self.geometry("1420x860")
        self.protocol("WM_DELETE_WINDOW", self.on_closing)

        # ĐƯỜNG DẪN MODEL CỦA BẠN đã train nhé
        MODEL_PATH =
r"runs\detect\runs\detect\yolo11m_v1\weights\best.pt" #
        if not os.path.exists(MODEL_PATH):
            # Tự động tìm xem có file .pt nào sẵn trong thư mục gốc
không
            if os.path.exists(r"weights\best.pt"):
                MODEL_PATH = r"weights\best.pt"
            elif os.path.exists("yolo11m.pt"):
                MODEL_PATH = "yolo11m.pt"
            elif os.path.exists("yolo11n.pt"):
                MODEL_PATH = "yolo11n.pt"

        self.engine = YOLOEngine(
            model_path=MODEL_PATH,
            cb_stats=self.on_stats,
            cb_log=self.on_log,
            cb_stop=self.on_stop,
            cb_error=self.on_error
        )
```

```
        self.dashboard = ProDashboard(self, self.engine)
        self.dashboard.pack(fill="both", expand=True)

        if self.engine.load_error:
            self.after(500, lambda: messagebox.showwarning(
                "Chú ý Đường dẫn Model",
                f"Không tìm thấy model tại:\n{MODEL_PATH}\n\nHãy vào mục '⚙ Cài đặt AI & Model' ở góc trái để chọn lại file best.pt của bạn!"
            ))

    # Vòng lặp lấy hình ảnh định kỳ mỗi 25ms (Không bao giờ làm treo giao diện)
    def start_ui_polling(self):
        if self.engine.is_running:
            if self.engine.latest_frame is not None and not self.engine.is_paused:
                self.dashboard.render_frame_on_canvas(self.engine.latest_frame)
                model_name = os.path.basename(self.engine.model_path)
                self.dashboard.lbl_status.configure(
                    text=f"FPS: {self.engine.current_fps:.1f}  |  Conf: {self.engine.conf_thresh:.2f}  |  Model: {model_name}"
                )
            self.after(25, self.start_ui_polling)

    def on_stats(self, count_in, count_out, add_in, add_out):
        def update():
            self.dashboard.lbl_in.configure(text=str(count_in))
            self.dashboard.lbl_out.configure(text=str(count_out))
            inside = max(0, count_in - count_out)
            self.dashboard.lbl_inside.configure(text=f"Đang ở bên trong: {inside} người")
            # Tự động cộng dồn vào file thong_ke_ngay.csv và cập nhật bảng + biểu đồ ngày
            self.dashboard.update_daily_csv(add_in, add_out)
        self.after(0, update)

    def on_log(self, track_id, event, conf):
        def add():
            now = datetime.datetime.now()
            d_str = now.strftime("%d/%m/%Y")
            t_str = now.strftime("%H:%M:%S")
            self.dashboard.tree.insert("", 0, values=(t_str, f"ID {track_id:02d}", event))
            self.dashboard.log_data.append({
                "Ngày": d_str,
```

```
                    "Thời gian": t_str,
                    "Mã Định Danh": int(track_id),
                    "Hành Động": event,
                    "Độ tin cậy": conf
                })
            self.after(0, add)

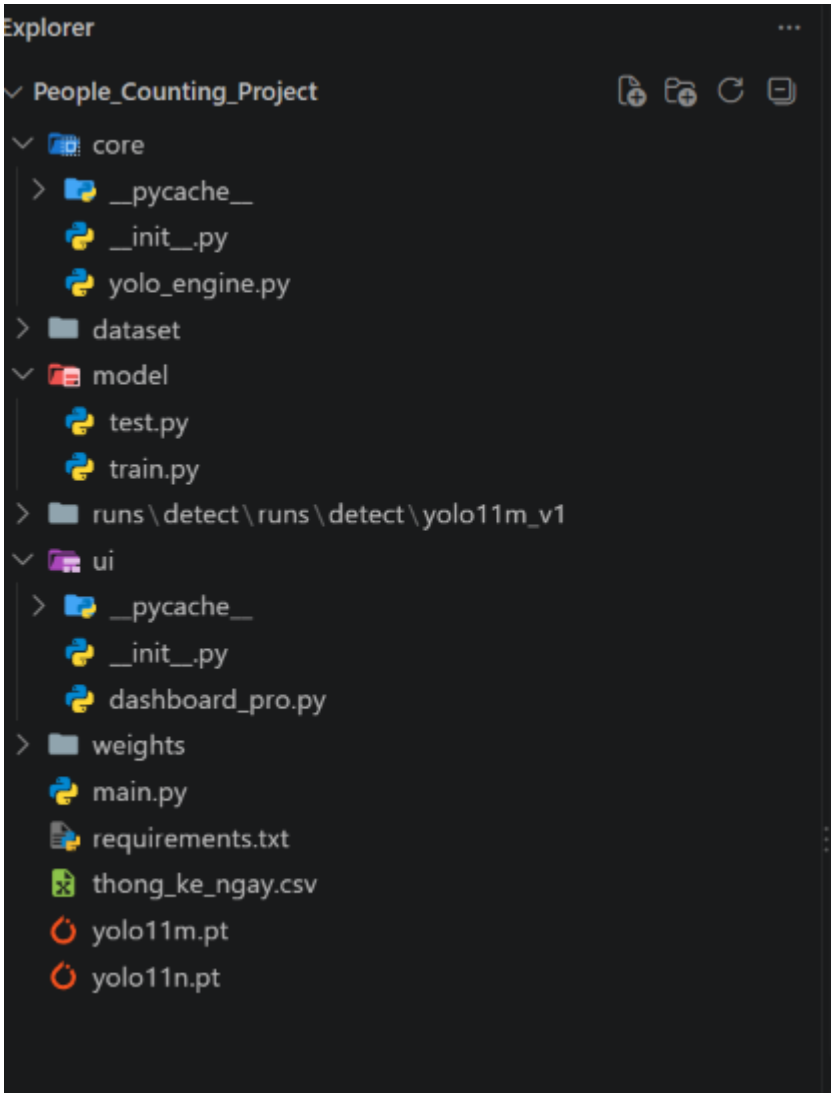
    def on_stop(self):
        def reset_ui():
            self.dashboard.btn_play.configure(text="▶ Bắt đầu",
fg_color="#0D6EFD")
            self.dashboard.btn_pause.configure(state="disabled",
text="⏸ Tạm dừng", fg_color="#F1F5F9", text_color="#2B3445")
            self.dashboard.btn_select.configure(state="normal")
            self.dashboard.lbl_cam_state.configure(text="◯ Đã kết thúc
phiên", text_color="#475569", fg_color="#F1F5F9")
        self.after(0, reset_ui)

    def on_error(self, err_msg):
        self.after(0, lambda: messagebox.showerror("Báo lỗi Hệ thống",
err_msg))

    def on_closing(self):
        self.engine.stop()
        self.destroy()

if __name__ == "__main__":
    app = App()
    app.mainloop()
```

## GIAI ĐOẠN 5: VẬN HÀNH VÀ SỬ DỤNG PHẦN MỀM THỰC TẾ




**Tạo file *__init_ _ .py cho 2* 📁 *ui và core còn file pycache khi chạy nó tự tạo***

Sau khi chép file best.pt vào thư mục phần mềm và sửa đường dẫn trong main.py, bạn vận hành như sau:

1. **Khởi động:** Chạy lệnh python main.py trong terminal.
2. **Nạp Video/Webcam:** Bấm vào nút 📁 *Chọn video* ở thanh điều khiển (hoặc mở menu bên trái để chọn Webcam).
3. **Kẻ vạch đếm:** Khung hình hiện ra, click chuột trái **2 điểm** lên vị trí cổng hoặc lối đi trên màn hình để giăng một sợi dây ảo (sẽ có mũi tên màu xanh chỉ hướng VÀO xuất hiện).
4. **Chạy AI:** Bấm nút ▶ *Bắt đầu*. Lúc này BoT-SORT sẽ hoạt động. Bất cứ ai cắt ngang vạch sẽ bị lưu ID, chớp sáng màn hình, cộng số liệu lên thẻ KPI và biểu đồ, đồng thời ghi log vào bảng lịch sử.
5. **Xuất báo cáo:** Cuối ngày, bấm nút 📄 *Xuất báo cáo*, phần mềm sẽ tự động gom toàn bộ file thong_ke_ngay.csv và chuyển thành file Excel (.xlsx) để báo cáo cho cấp quản lý.